TEAM GLITCH

ENGINEERING JOURNAL

WRO FUTURE ENGINEERS SEASON 2026

TEAM: ZEUS WADIA
SHAURYA SULE
REHAAN DHANDHIA

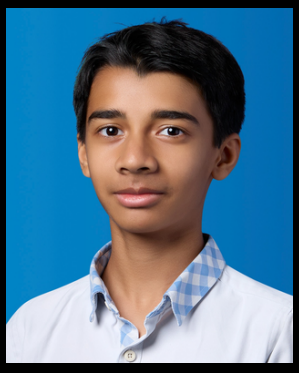
TEAM MEMBERS

Electronics, Testing, Mechanical Design

Zeus Wadia is a Grade 9 student at DAIS with a strong interest in STEM and robotics. Outside robotics, he enjoys playing and following sports.

Vision, Electronics, Mechanical Design

Shaurya Sule is a Grade 9 student passionate about robotics, STEM, and humanities. He also enjoys debating and sports.



Software, Vision & Sensor Integration

Rehaan Dhandhia is a Grade 9 student at DAIS, passionate about programming and robotics. He also enjoys badminton and playing the piano.

Check Out Our Team Links

Our Github Repository

Our Youtube Channel

ROBOT KEY SPECIFICATIONS

Parameter	Value
Dimensions	190 x 120 x 295 mm
Weight	1.35 kg
Drive Type	Rear-wheel differential drive with 12V DC motor
Steering	Servo-actuated parallel steering
Primary Brain	Raspberry Pi 4 Model B (4 GB)
Co-processor	Seeed XIAO ESP32-C3 (IMU + encoder over USB UART)
Vision	HikVision DS-UO2 USB Camera + OpenCV HSV pipeline
Heading	Adafruit BNO085 9-DOF IMU (on XIAO, I2C)
Battery	LiPo 11.1V, 850mAh, 30C
Primary Programming Language	Python
Secondary Programming Language	C++
Max Operating Speed	1.04 m/s

LABELLED FRONT VIEW

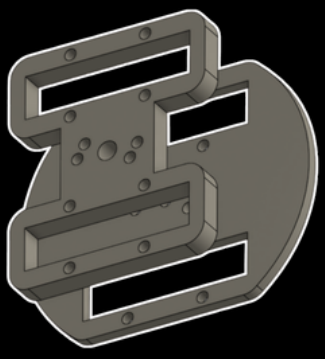
Vikram 453R6
Motor Driver

BNO085 IMU

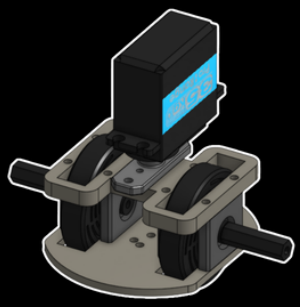
HikVision USB
Webcam

Johnsons 900 RPM
12V DC Motor

35kg-cm Servo

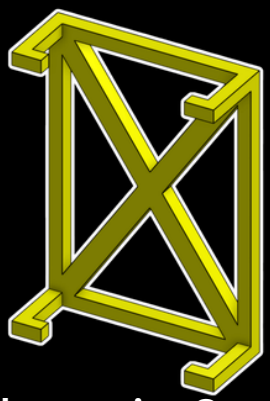


Mild Steel Deadweight

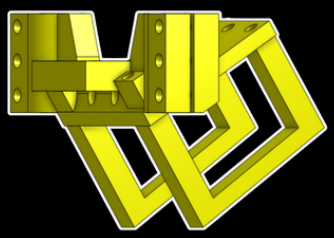


Parallel servo
steering

LABELLED BACK VIEW

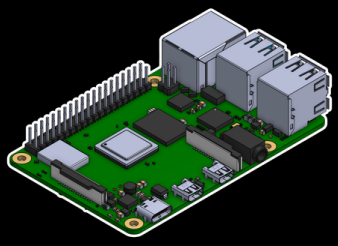


Electronics Cover

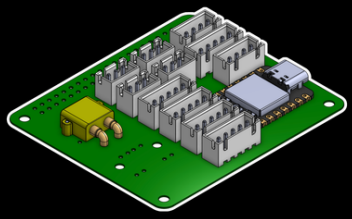


3D printed clamped camera mount

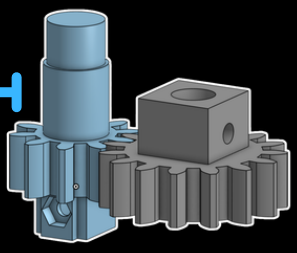
11.1V 850mAh LiPo Battery



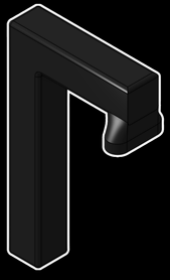
Raspberry Pi 4B



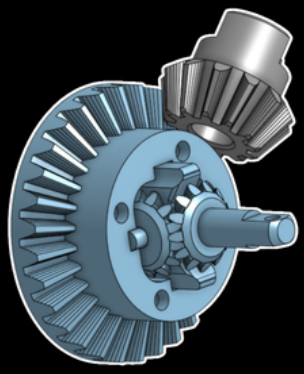
Custom PCB



Secondary Gearbox (Gear Reduction)



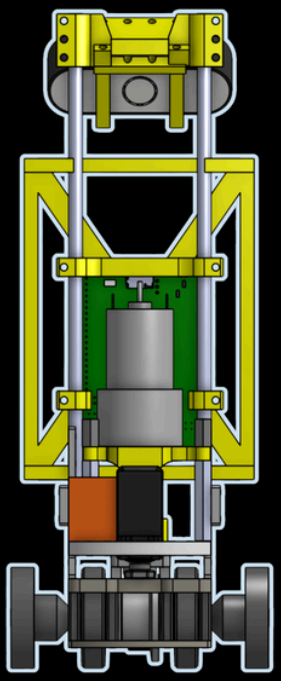
Rear Pivot Restrictor



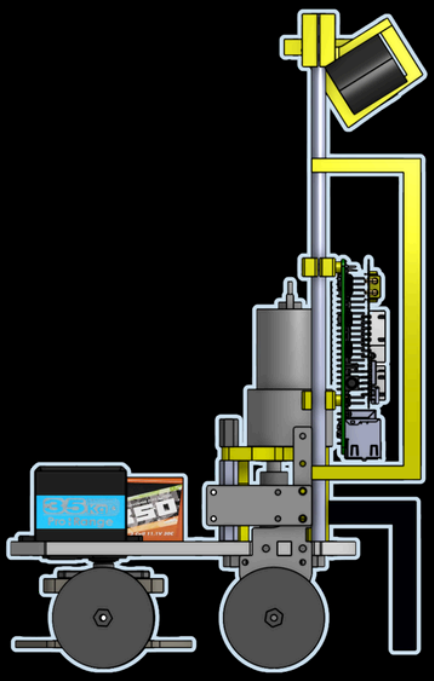
1:3 Metal Differential Gearbox

ROBOT CAD VIEWS

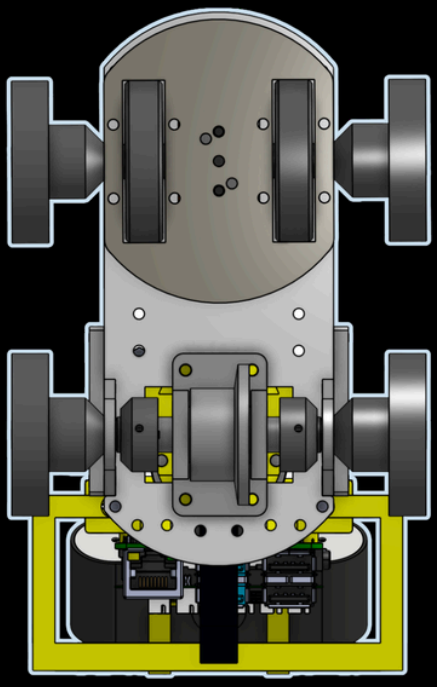
Front View:



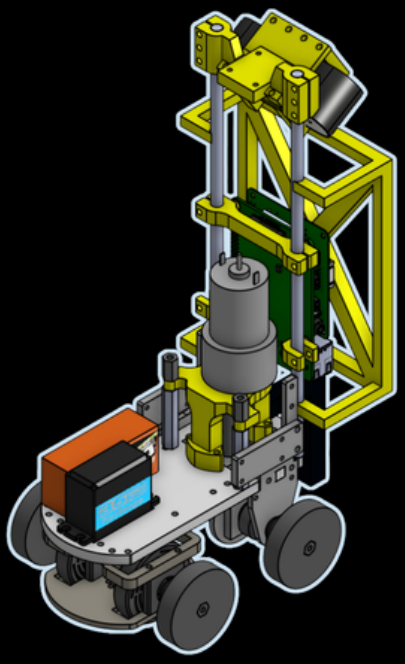
Side View:



Bottom View:



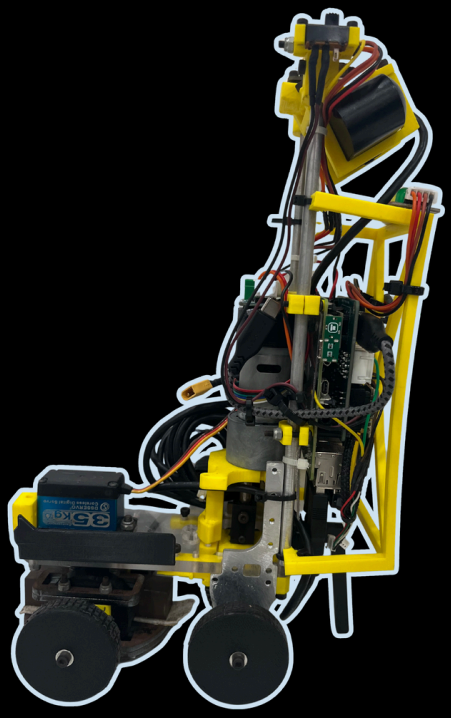
Isometric View:



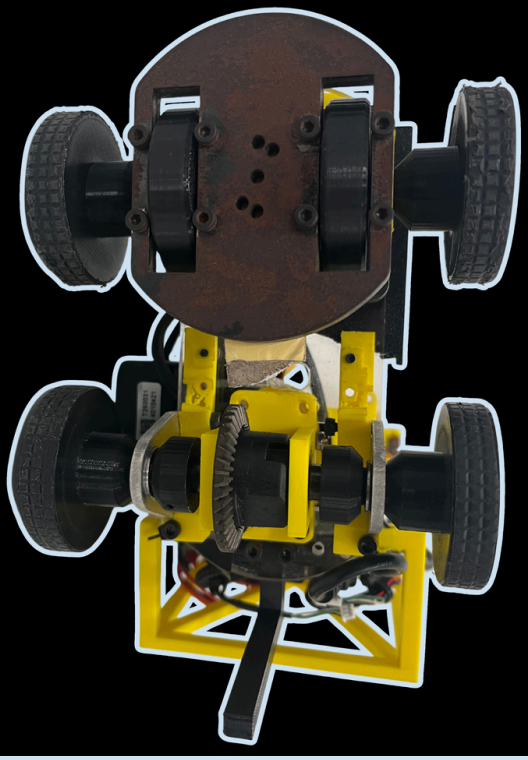
ROBOT REGULAR VIEWS

Front View:

Side View:



Bottom View:



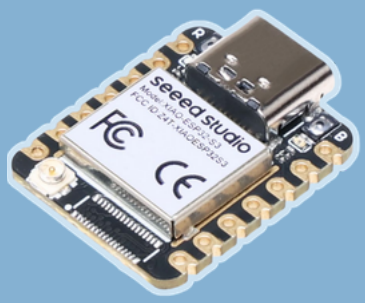
Isometric View:

ONBOARD ELECTRONICS

Raspberry Pi 4B (4 GB)

- Quad-core Cortex A72 processor, 4 GB LPDDR4 RAM, 40 GPIO pins, USB 3.0, 2x Micro-HDMI ports
- Primary microcontroller, Vision compute, sensor control, PID control via PIGPIO.

Seeed XIAO ESP32C3:



- 160 32-bit RISC-V single core (processor), 400 KB SRAM, 11 GPIO I/O pins
- Co-processor, controls BNO085 IMU, external encoder, UART communication with Pi (15200 baud rate).

TF Mini Plus LiDAR

- 0.1-12 meter range, ±5cm accuracy, 3.6° FOV, 5V, UART capability (152000 bps)
- Used to detect distance from wall (on right-side of bot) during parking sequence.

Adafruit BNO085 IMU

- 1.7-3.6V, 9 Degrees of Freedom
- Used to identify heading (used for PID steering).

ONBOARD ELECTRONICS

HikVision DS-UO2 Webcam

- 1920x1080 resolution, 30 FPS, Wide FOV, USB 2.0
- Main source of input - robot is mainly vision based. Gives input to OpenCV HSV Pipeline.

Johnsons 900 RPM DC Motor

- 900 RPM, 1.2 kg · cm rated torque, 800mA rated current
- Used for rear wheel drive via Vikram 453R6 motor driver.

DS3235 35 kg · cm Servo

- 0.18 sec / 60° (at 5V), 23 kg · cm torque (at 5V).
- Controls parallel steering (at 5V).

Vikram-453R6 Motor Driver

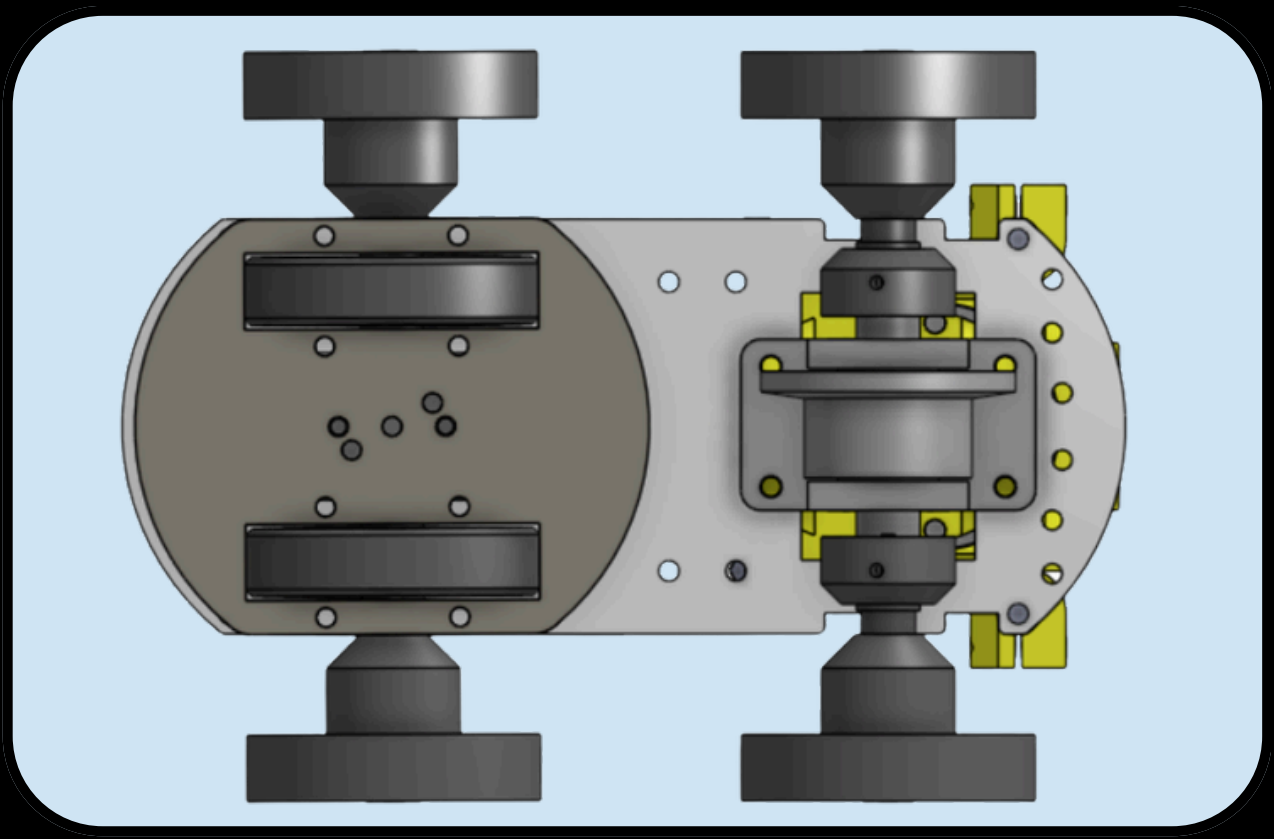
- Single channel H bridge, 6.5-45V DC, 2A continuous output, 3.6A output.
- Motor driver for Johnsons DC Motor, auto-recovery on overcurrent.

Custom PCB

- Custom designed using Altium designer, contains JST 2.54 mm pins and USB ports.
- Simplifies wiring, makes sure no wires are loose

SECTION 1

Mobility & Mechanical Design



Points Covered:

- Chassis design choices
- Steering mechanism justification
- Drive mechanism justification
- Torque & speed reasoning
- Mechanical stability

Points Covered: Chassis design choices (Section 1), Mechanical stability (Section 1)

CHASSIS DESIGN

- **Lightweight & Compact**: We have made the robot chassis as small as possible, which minimizes unnecessary weight and keeps it maneuverable in tight spaces.
- **Rigid & Sturdy**: A firm structure provides stability and securely mounts all subsystems for reliable performance.
- **Agility & Durability**: The balance of strength and low weight enables fast movement, smooth handling, and consistent performance throughout the competition.
- **High Stability:** Even though the camera and motor are at the back, the robot retains stability due to a deadweight at the front and a topple restrictor at the back

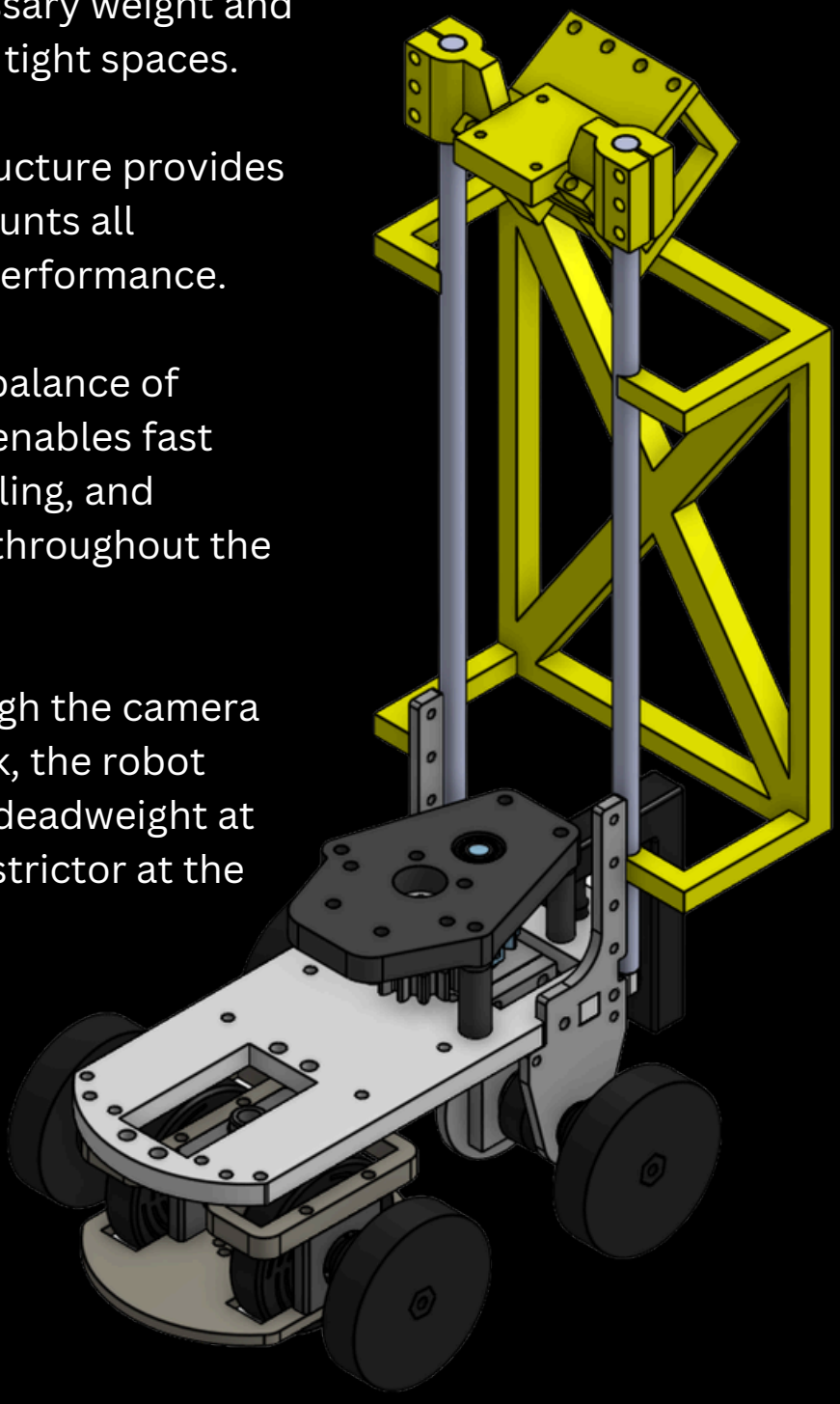


Figure 1.1 - Robot Skeleton

Points Covered: Drive mechanism justification (Section 1), Engineering reasoning for decisions (Section 4)

DRIVE MECHANISM

We use a **Johnson 900 RPM 12V** motor with an off-the-shelf differential gearbox to drive the rear wheels.
A differential gearbox, using a set of 4 perpendicular inner gears, 1 large ring gear, and 1 small bevel gear, allows the 2 output axles to rotate at different speeds. This is particularly important while turning, where each wheel has a different turn radius. Forcing the same angle onto both would result in tyre scrub, slippage, and inefficient power distribution. Figure 1.2 shows the manufacturer's CAD.

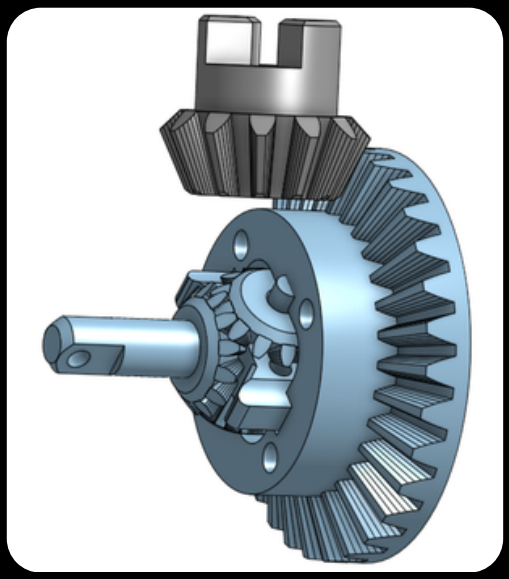


Figure 1.2 - Differential CAD

Why We Chose This:

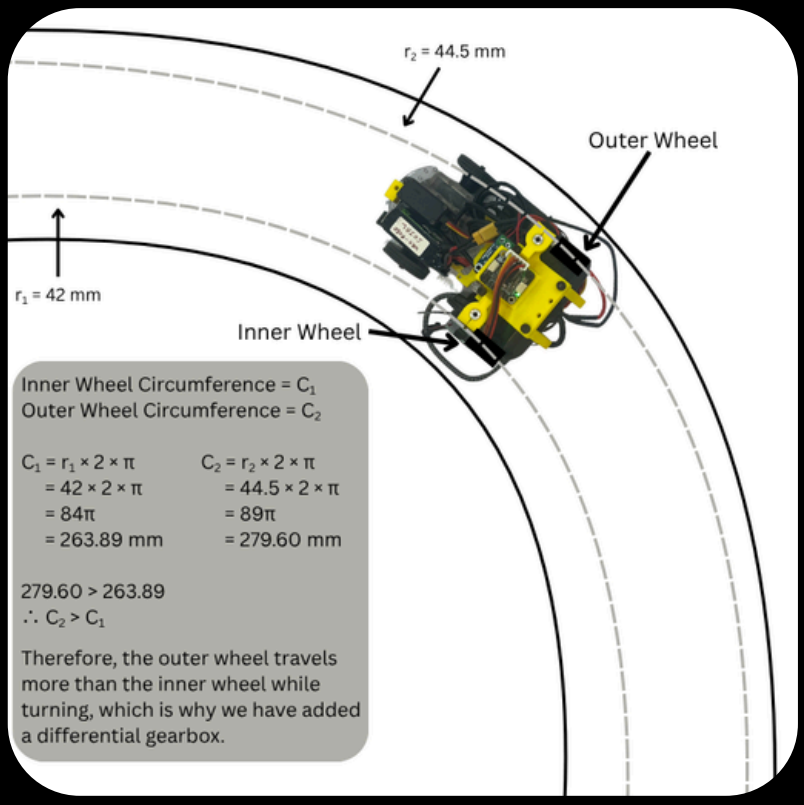


Figure 1.3 - Reason for Differential Gearbox

As shown in Figure 1.3, during a turn, the outer wheel travels a larger radius than the inner wheel. The differential allows the wheels to rotate at different speeds, reducing power loss and wheel slippage and keeping the robot stable.

Points Covered: Steering mechanism justification (Section 1), Engineering reasoning for decisions (Section 4), Constraints & trade-offs (Section 4), Iteration & testing cycles (Section 4)

STEERING MECHANISM

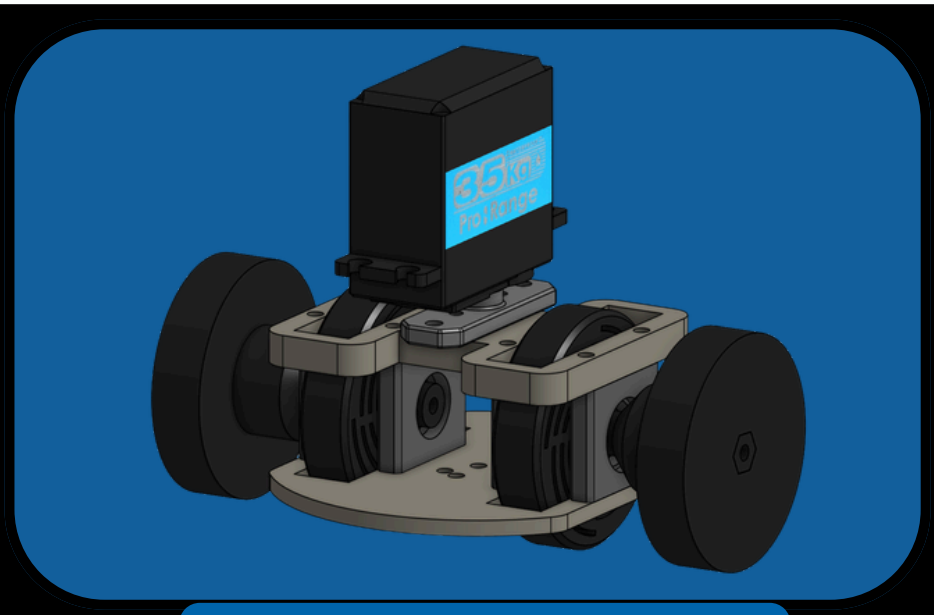


Figure 1.4 - Parallel Steering Linkage

We use a 35 kg servo for steering, connected to a servo horn and a laser-cut steel plate with 3D-printed wheel mounts. The servo provides up to 90° steering for sharp and wide turns.
We tested Ackermann steering (which turns wheels differently according to turn radius), but it was extremely sensitive to loose parts and slipped at higher speeds. This caused unpredictable steering and higher power use, so we chose to use a parallel steering linkage instead.

Acknowledging Trade-Offs:

- **Benefits of Parallel:** Much simpler and more straightforward hardware
- **Drawbacks of Parallel:** Increases weight and tyre scrub at lower speeds. FIXED by adding cat-tongue tape
- **Benefits of Ackermann:** Much more compact and light, aligns wheels to turn radius
- **Drawbacks of Ackermann:** Very sensitive to mechanical play and causes skidding at higher speeds

Points Covered: Steering mechanism justification (Section 1), Torque/speed reasoning (Section 1), Engineering reasoning for decisions (Section 4), Constraints & trade-offs (Section 4), Iteration and testing cycles (Section 4)

SERVO CHOICE REASONING...

Analysing Servo Motor Choices

Figure 1.5 - MG996R

Figure 1.6 - GS2010MG

Figure 1.7 - DS3235

Servo Option	Speed (At 5V)	Stall Torque (At 5V)	Verdict
MG996R Servo	0.19 sec / 60°	9.4 kg · cm	Low torque, slow speed. Not ideal choice.
GS2010MG 20kg · cm Servo	0.12 sec / 60°	21 kg · cm	Very high speed, good torque.
DS3235 35kg · cm Servo	0.18 sec / 60°	23 kg · cm	**Current servo choice. Very high torque, slightly slow.**

Acknowledging Trade-Offs:

The DS3235 35kg · cm servo has a lower speed than the GS2010MG Servo (by 0.06 sec / 60°). However its torque (at 5V) is higher than the GS2010MG's by 2 kg · cm. Therefore, overall, we found this change beneficial, reducing risk of the servo failing (due to high load).

TORQUE CALCULATIONS

As shown in Figure 1.8, we calculated our minimum torque requirement for our motor. Estimating a rolling resistance coefficient of 0.03 (everything else was measured), we found that our motor would need around 0.185 kg·cm of torque (considering the 1:2 gear ratio).

Minimum Torque Calculations:

Mass of Robot = 1.35 kg

Desired Acceleration = 1.0 m/s²

Net Force = Mass × Acceleration

= 1.35 × 1.0

= **1.35 N**

Gravitational Field Strength = 9.8 N/kg = 9.8 m/s²

Rolling Resistance Coefficient = 0.03

∴ Resistant Force = 9.8 × 0.03 = **0.294 N**

Driving Force = Net Force + Resistant Force

= 1.35 + 0.294

= 1.644 N = 0.168 kgf

∴ Required Driving Force = **0.168 kgf**

Wheel Radius = 2.2 cm

Required Drive Torque = Force × Radius

= 0.168 × 2.2

= **0.3696 kgcm**

Total Gear Ratio = (1.5:1) × (1:3) = 1:2

∴ Required Motor Torque = Torque ÷ 2

= 0.3696 ÷ 2

= **0.185 kg cm**

Figure 1.8 - Torque Calculations

Points Covered: Torque/speed reasoning (Section 1), Drive mechanism justification (Section 1), Current draw reasoning (Section 2), Engineering reasoning for decisions (Section 4), Constraints & trade-offs (Section 4), Iteration & testing cycles (Section 4)

DC MOTOR CHOICE REASONING

Motor Option	RPM	Rated Torque (kg · cm)	Rated Current (mA)	Verdict
Cytron IG42E-04K Planetary DC Motor	1400	2.24	**5500**	We considered this motor as it had high torque and high speed. However its rated current of 5500 mA exceeded our motor driver's max current of 2000 mA
Rhino GB37 1000RPM	1000	**0.17**	300	We used this motor in our first bot iteration, but with a torque of only 0.04 kg · cm above our minimum torque requirement (which at that time was 0.13 kg · cm without secondary gearbox), this motor was inconsistent and often failed.
Johnson 300 RPM Grade A	**300**	3.6	800	We considered using this motor due to its high torque and low current requirement, but after calculating the operational speed it would give our bot (0.28 m/s), we realised it would not be fast enough.
Johnson 900 RPM Grade A	900	1.2	800	**The current motor we are using. Has a much better balance of speed and torque**

Points Covered: Torque/speed reasoning (Section 1), Drive mechanism justification (Section 1), Engineering reasoning for decisions (Section 4), Constraints & trade-offs (Section 4)

SPEED AND TORQUE REASONING

While Choosing our DC Motor

- While choosing our DC motor, we tried to **prioritise torque** over speed to make our bot more consistent.
- To do this, we use a Johnsons DC motor with 900 RPM and 1.2 kg · cm of torque.

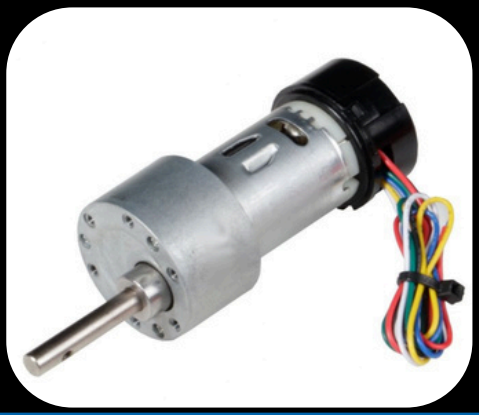


Figure 1.9 - Johnsons DC Motor

Adding a Second Gearbox (For Speed Increase)

- During the National Championship, we realised that our bot was not very fast (max no-load speed of 0.69 m/s).

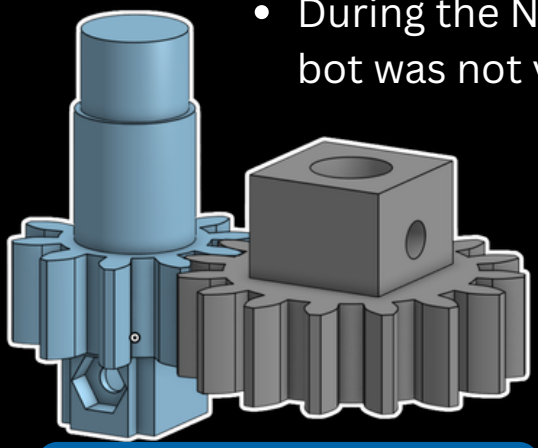


Figure 1.10 - Second Gearbox

- While wishing to increase this, we did not want to change our motor (since it was very consistent) or alter the differential gearbox (since it was store bought).
- We therefore decided to add a second gearbox (connecting the motor to the differential gearbox) with the gear ratio 1.5:1.
- This increased the speed by 50%, reducing torque by 33.3%.

Overview of Speed & Torque Reasoning

- We overall tried to prioritise torque over speed for increased consistency.
- However, observing excessive torque, we attempted to increase speed as much as possible, while still prioritising high torque.

SPEED CALCULATIONS

- The calculations in figure 1.11 helped us estimate the robot's no-load speed (in use) before physical testing.
- By accounting for the motor RPM, gearbox reduction, and wheel circumference, we obtained a **no-load maximum speed of 1.04 m/s**.
- However, we usually run our motor at 80% power, therefore we calculated a **theoretical operational speed of 0.83 m/s.**

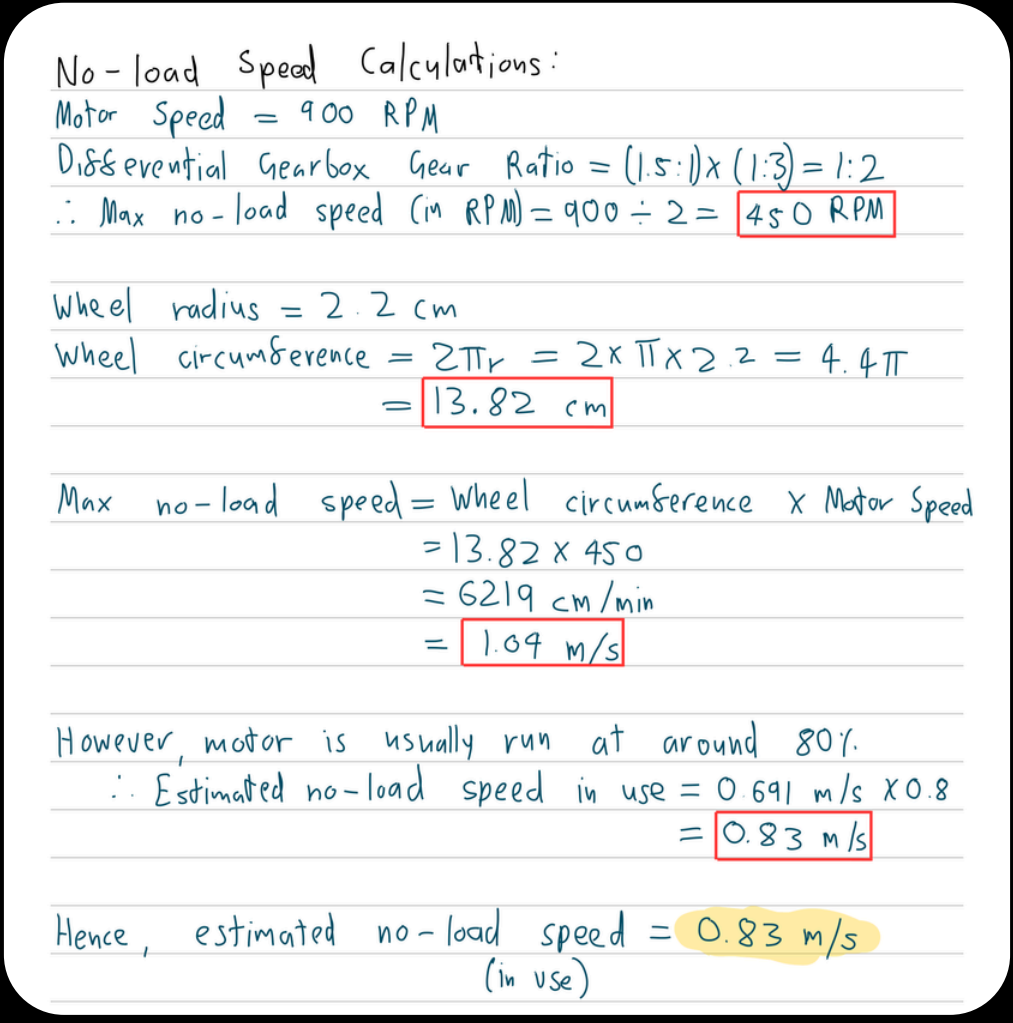


Figure 1.11: Speed Calculations

Points Covered: Mechanical stability (Section 1), Chassis design choices (Section 1), Sensor placement justification (Section 2), Engineering reasoning for decisions (Section 4), Constraints & trade-offs (Section 4), Risk identification & mitigation (Section 4)

STABILITY: DEADWEIGHT

Adding a Deadweight

- For our robot's stability, we focused on having a stable robot while allowing it to get the best vantage point for our camera as well.
- We needed our camera to be mounted high, and we realised that creating a large mount with a lot of support across the robot would not only be resource inefficient but also heavier than intended.
- Hence, we chose to leave the camera mount as it was, and a deadweight in the front such that the robot does not topple over.

Acknowledging Trade-offs

- Increased mass of robot by around 100 grams.
- Reduced max speed due to mass increase (~100 grams).

Change in Center of Mass

- Figure 1.12 compares the center of mass of our robot with and without the deadweight
- Note that this diagram is generated by and exported from Onshape based on our robot's CAD (all materials and dimensions set accurately).

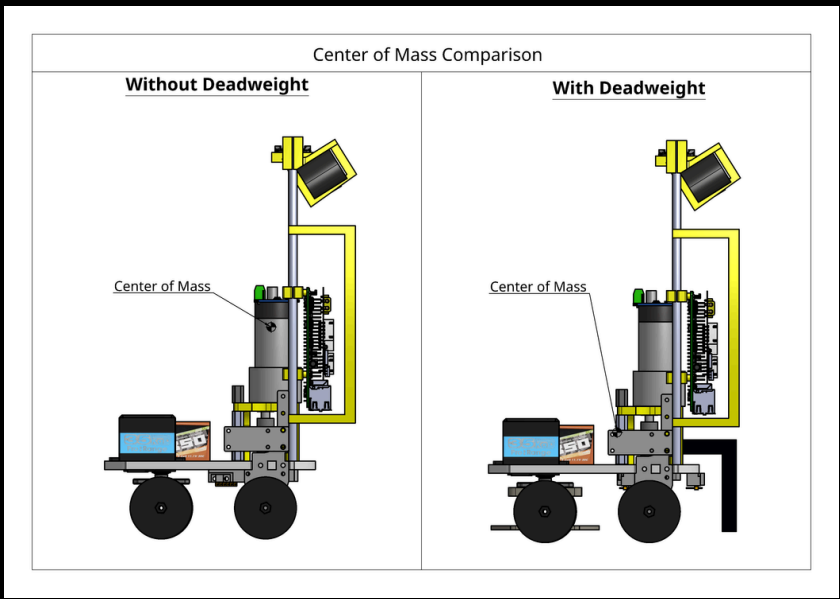


Figure 1.12 - Center of Mass Comparison

Points Covered: Mechanical stability (Section 1), Chassis design choices (Section 1), Engineering reasoning for decisions (Section 4), Constraints & trade-offs (Section 4), Risk identification & mitigation (Section 4)

REAR PIVOT RESTRICTOR

Risk Mitigation:

To decrease risk of our robot toppling, we decided to add a rear pivot restrictor on the rear end of our robot. This ensures that if our robot loses balance falls backward, it does not fully topple over. This improved consistency.

Not Damaging the Mat

To ensure that this did not damage the mat during a run, we ensured that the pivot restrictor is ~7 mm above the contact between the wheel and the mat, preventing it from dragging across and potentially damaging the mat.

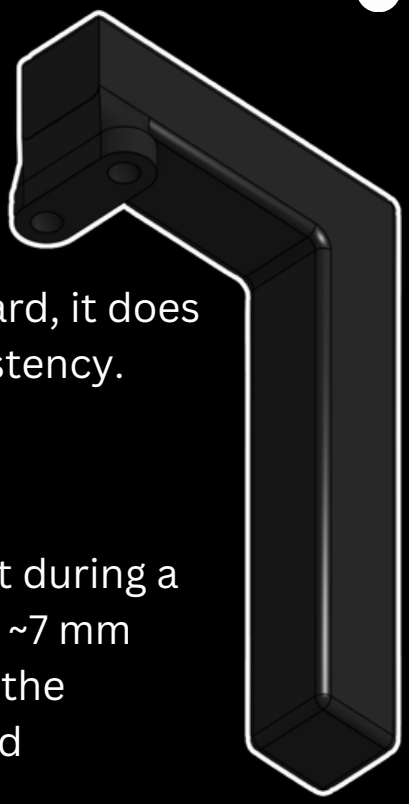


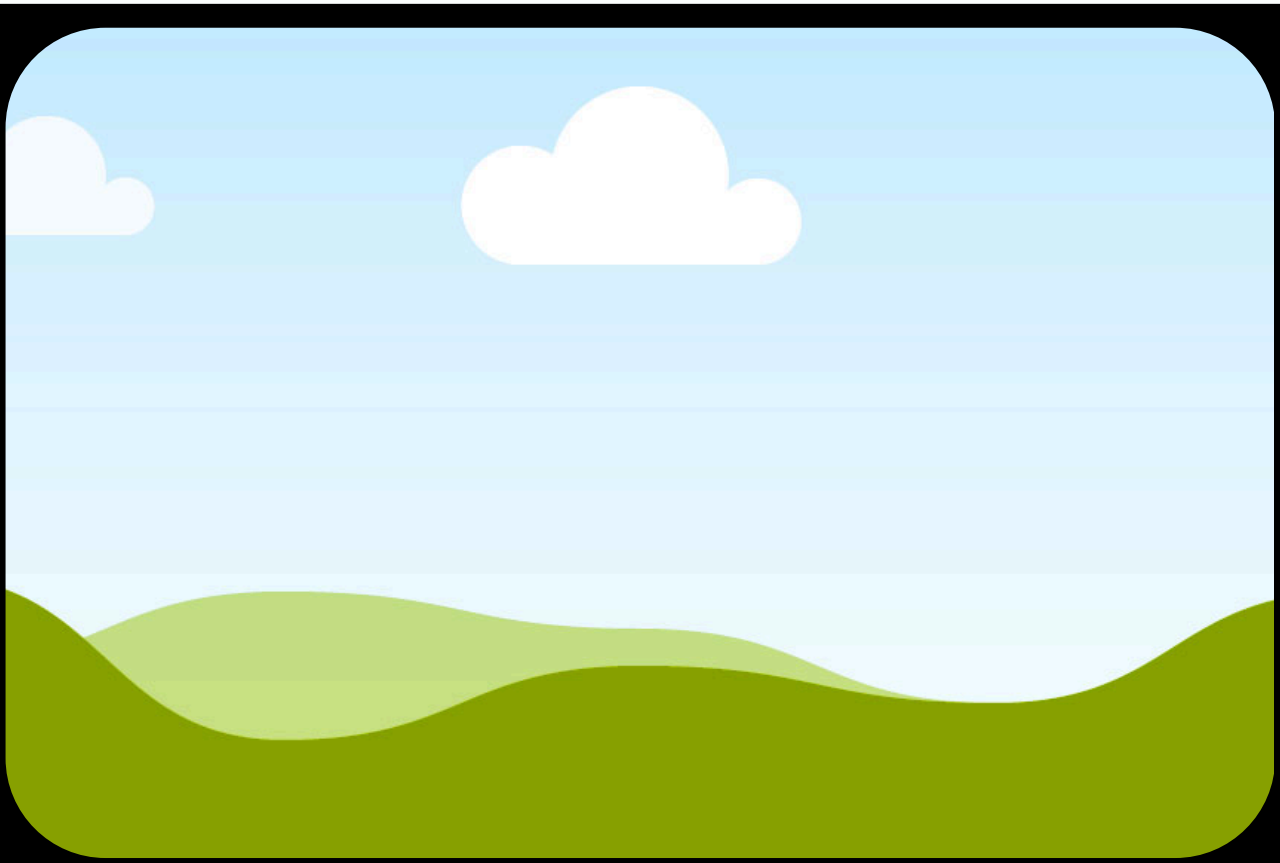
Figure 1.13 - Rear Pivot Restrictor

Acknowledging Trade-Offs

- One drawback of using this pivot restrictor is that it moves the center of mass of the robot further towards the rear end of our robot (where our robot was already back heavy).
- However, being made of PLA, it only had a mass of 11.5 grams, making the change in center of mass almost negligible.
- Overall, using the rear pivot restrictor helped safeguard our runs, mitigating risk of our robot toppling over.

SECTION 2

Power & Sensor Architecture



Points Covered:

- Power system architecture
- Current draw reasoning
- Sensor selection justification
- Sensor placement justification
- Calibration
- Wiring diagrams

Points Covered: Wiring diagrams (Section 2)

WIRING DIAGRAM

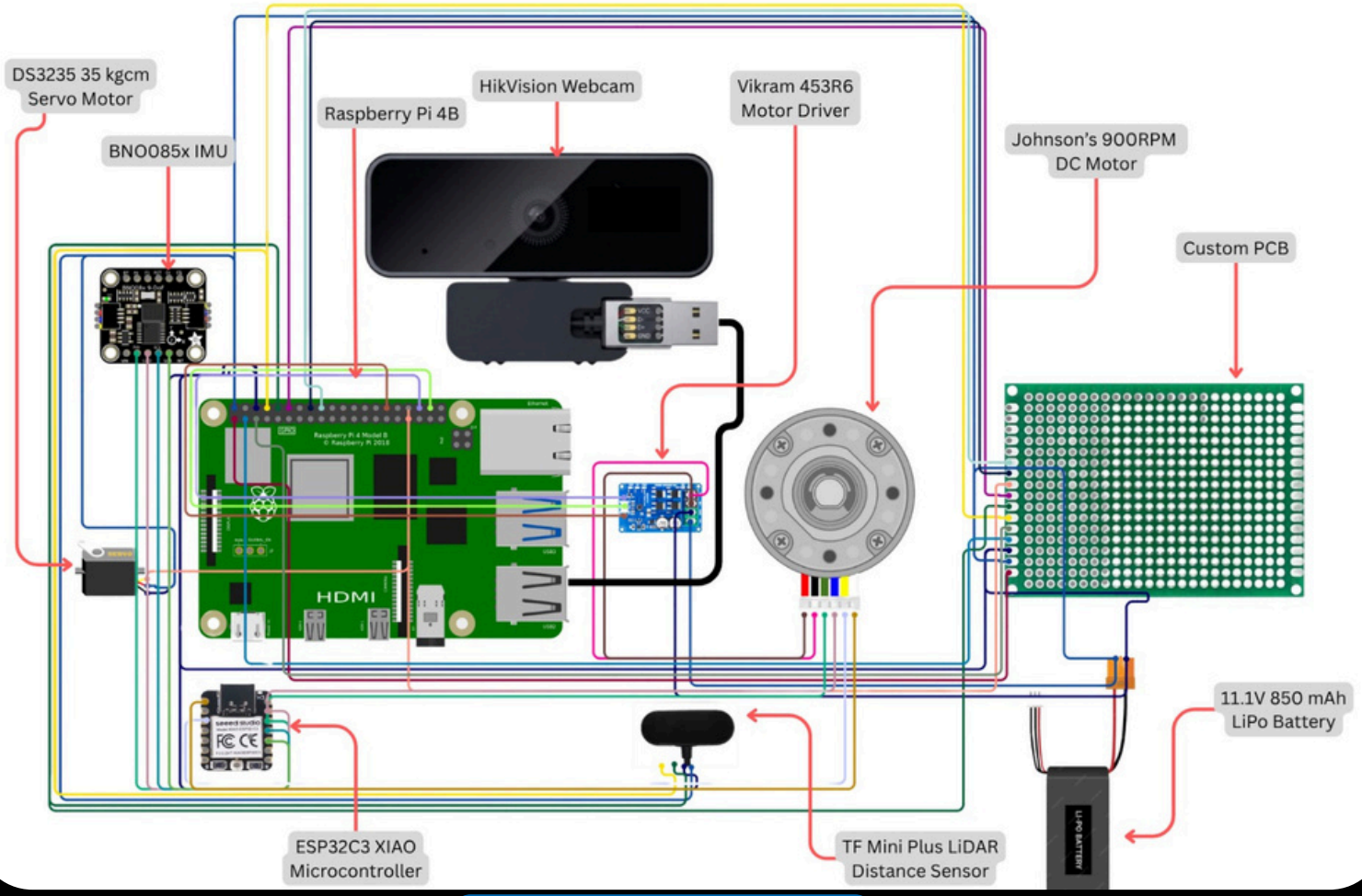
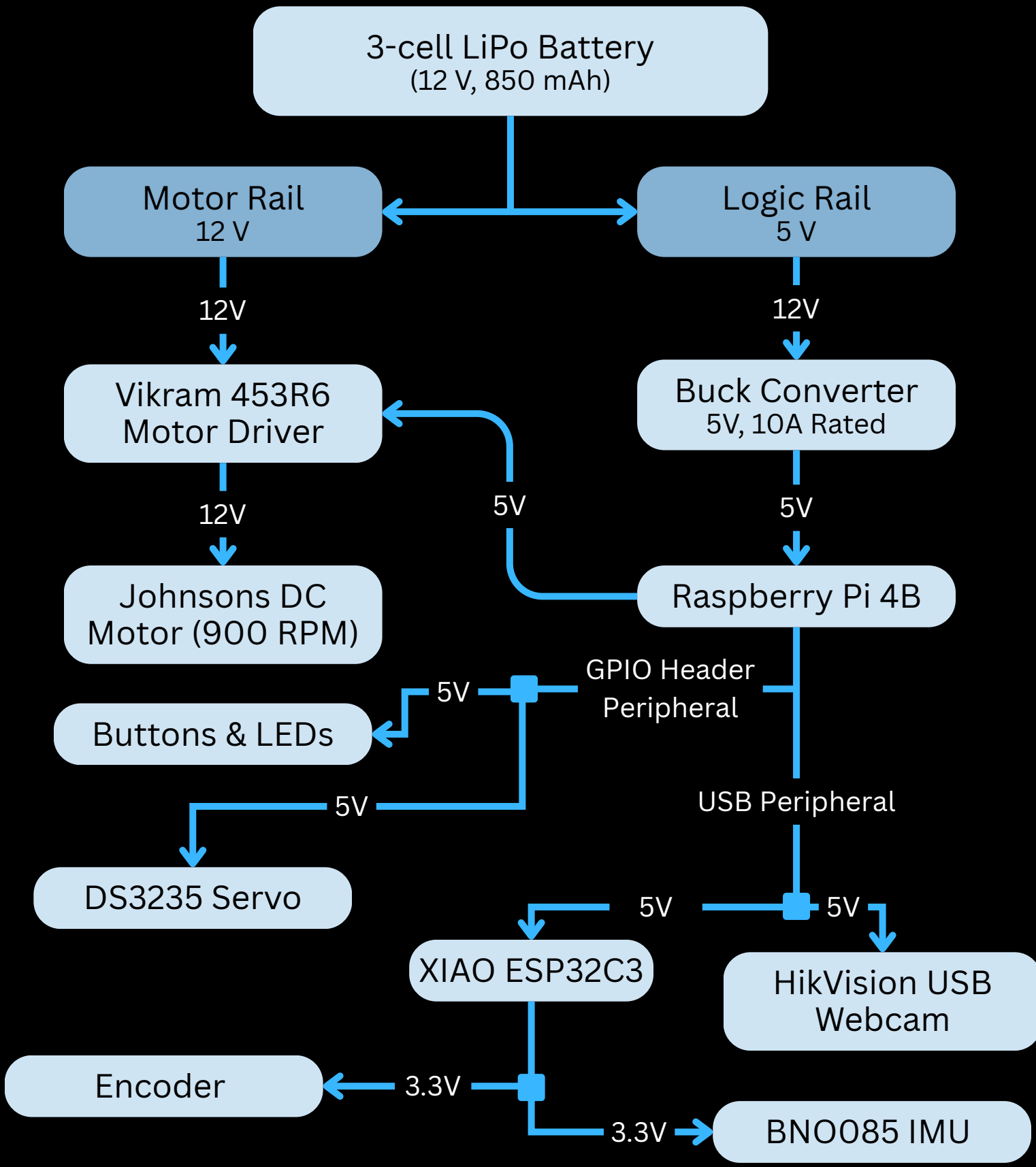


Figure 2.1 - Wiring Diagram

POWER RAILS


3-cell LiPo Battery
(12 V, 850 mAh)
Motor Rail
12 V
Logic Rail
5 V
12V
12V
Vikram 453R6
Motor Driver
Buck Converter
5V, 10A Rated
12V
5V
5V
Johnsons DC
Motor (900 RPM)
Raspberry Pi 4B
GPIO Header
Peripheral
5V
Buttons & LEDs
5V
USB Peripheral
DS3235 Servo
5V
5V
XIAO ESP32C3
HikVision USB
Webcam
Encoder
3.3V
3.3V
BNO085 IMU

POWER RAILS

Power Rails: Overview

- **12V LiPo Supply:** Our battery powers the system at 12V.
- **Dual-Rail System:** The robot uses separate power rails for the motor, and the other electronics.

- **Motor Rail (12V):** The motor rail is directly powered by the battery at 12 volts. The battery gives 12 volts to the motor driver (Vikram 453R6), which then powers the DC motor directly at 12V.

- **Electronics Rail (5V):** The battery provides 5V to the Raspberry Pi via a buck converter (which regulates the circuit down to 5V).
  - **GPIO Header Peripheral:** Via the Raspberry Pi's GPIO pins, the DS3235 Servo, buttons and LEDs are given 5V.
  - **USB Peripheral:** Via the Raspberry Pi's USB ports, the Hikvision USB Camera and XIAO ESP32C3 are given 5V.
    - **3.3V Sensors:** The XIAO then regulates it down to 3.3V, powering the BNO085 IMU and encoder at 3.3V.

Benefits of Dual-Rail System

- **Noise Isolation:** If on a single rail, electromagnetic interference (electrical noise) can cause sensor glitches and innacuracies in sensor readings. Separating the motor rail and logic rails prevents this.

Points Covered: Power system architecture (Section 2), Engineering reasoning for decisions (Section 4), Risk identification and mitigation (Section 4)

CUSTOM PCB

The Problem

- Originally, our wiring was extremely messy, as we had single jumpers coming out of the Raspberrry Pi, ESP32, and sensors. It also had large risks.
- It was also very loose and prone to frequent disconnections if the robot hit a wall or obstacle, and some wires would come out even when it was idle
- Rewiring or changing any modules was extremely impractical as you had to pick out exactly the right wires, without messing with any others

Our Solution

- We designed a custom PCB using Altium designer.
- On one side of our PCB, we have 2 rows of female pin headers, connected directly to the Raspberry Pi's GPIO pins.
- For power input (from the battery), we use an XT30 connector.
- To connect to most electronics (sensors, servo, motor driver), we use JST connectors which lock the wires, securing the connections.
- The XIAO ESP32C3 is attached to the the PCB through female pin headers.
- This PCB is then screwed directly onto 3D printed mounts on the rods.
- Overall, using this PCB helps us secure and organise our wiring, mitigating risks.

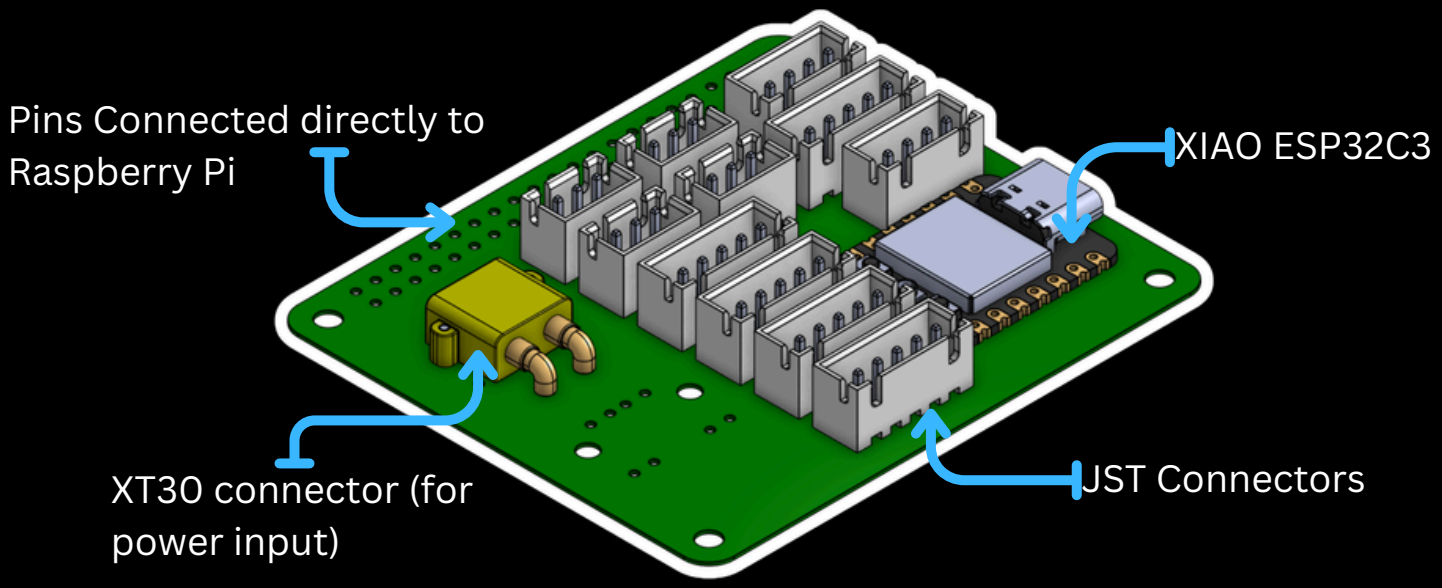


Figure 2.2 - PCB CAD + Labels

Points Covered: Power system architecture (Section 2), Current draw reasoning (Section 2)

POWER BUDGET ESTIMATE

- This is our power budget estimate for the entire robot, including sensors, actuators, and processors
- The highest consumer is the motor, with 12V 800mAh (9.6A) required while running
- Overall, the total power required is 29 watts, of 40mAh at 12V. Our battery - 12V 850mAh - can supply a sustainable 12V to the robot for at least 5 consecutive runs

Component	Voltage	Expected Current Draw	Expected Power
Johnson 900RPM DC Motor	12V	800 mA running (2400 mA stall)	9.6W (28.8W Stall)
Raspberry Pi 4B	5V	1200 mA	6.0W
XIAO ESP32C3	5V	200 mA	1.0W
DS3235 Servo	5V	1000 mA (1900 mA stall)	5.0W (9.5W Stall)
External Quadrature Encoder	3.3V	3 mA	0.001W
BNO085 IMU	3.3V	7.5 mA	0.025W
Total (Normal)	N/A	~3.2105 A	~21.63W

BATTERY RUNTIME CALCULATIONS

- Figure 2.3 shows our calculations for our battery's estimated runtime.
- As shown in figure 2.3, our battery can run for ~21 minutes.
- Note that the section labelled "Battery Specifications" gives specifications including voltage (nominal and full charge), capacity, and C-rating.
- Some specifications (like full charge voltage and C-rating) are not required for battery runtime calculations, but we have still provided them.

Battery Specifications:

Nominal Voltage = 11.1 V
Full charge Voltage = 12.6 V
Capacity = 850 mAh
C-rating (Discharge Rating) = 30C

Energy Capacity = Nominal Voltage × Capacity
= 11.1 V × 0.85 Ah
= 9.435 Wh

Discharge Limit = Capacity × C-rating
= 0.85 Ah × 30C
= 25.5 A

Battery Runtime Calculations:

Expected Power Draw = 21.63 W

$$\text{Max Expected Runtime} = \frac{\text{Energy Capacity}}{\text{Power drawn}} = \frac{9.435\ \text{Wh}}{21.63\ \text{W}} = 0.436\ \text{h}$$
= 26.2 minutes

Estimated Efficiency = 0.8 = 80%

Estimated Runtime = Efficiency × Max Runtime
= 26.2 min × 0.8
= 20.96 = 21 min

∴ Estimated Runtime = 21 minutes

Figure 2.3 - Battery Runtime Calculations

Points Covered: Sensor selection justification (Section 2), Engineering reasoning for decisions (Section 4), Constraints & Trade-offs (Section 4), Iteration and testing cycles (Section 4)

CAMERA CHOICE

Pixy2 Camera vs Hikvision Webcam

v1 - Pixy2

- The Pixy2 AI Camera has its own onboard processor that extracts colour coordinates and sends them to the processor.
- It had a high 60 FPS frame rate that allowed the robot to react quickly.
- It was a great starting point as it was small, lightweight, and featured easy built-in color detection.
- However, its sow resolution and high sensitivity to shadows/glare made color detection unreliable under different types of lighting, and calibration was extremely difficult.
- It was also not compatible with OpenCV

Figure 2.4 - Pixy2

v2 - Hikvision Webcam

- It had a much higher resolution (1080p) which vastly improved image clarity.
- It had a much wider field of view, which allows it to capture a larger area of the track to spot obstacles much earlier.
- It is fully compatible with OpenCV, which lets it give consistent color and adjustable exposure - made obstacle detection highly reliable anywhere.
- We could also program a fast calibration script with OpenCV, allowing the robot to quickly adapt to the field at competitions.
- However, it was much heavier than the Pixy2

Figure 2.5 - Hikvision Webcam

Points Covered: Sensor selection justification (Section 2), Engineering reasoning for decisions (Section 4), Constraints & Trade-offs (Section 4), Iteration and testing cycles (Section 4)

CAMERA CHOICE

Pixy2 Camera vs Hikvision Webcam

Feature	Pixy2	HikVision
Resolution	720 P	1080 P
FPS	60	30
FoV (degrees)	60	86
Weight	10g	50g
Lighting adaptability	Low	High
Detection precision	Colour blobs only	Contours, shading

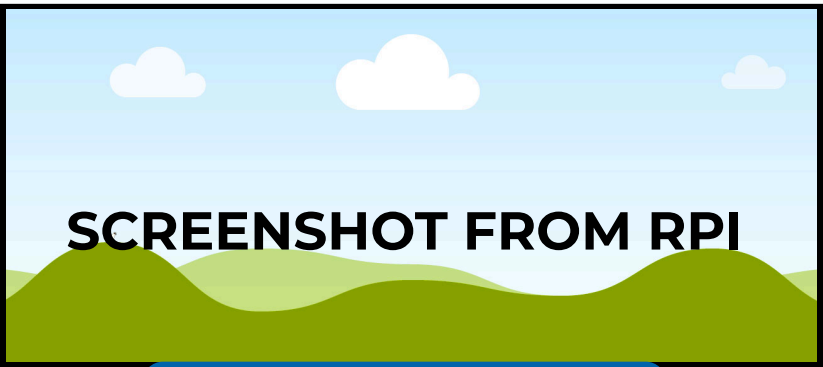


Figure 2.6 - RasPi Camera View

IMU CHOICE

MPU6050 vs Acrome BNO055 vs Adafruit BNO085

Our Goals while Choosing an IMU

- **Size:** We wanted our IMU to be compact and able to fit in a small slot (30mm X 25mm) on top of our camera mount.
- **Prioritising Gyroscope:** We wanted our IMU to have a very accurate gyroscope, as we are only using the gyroscope.

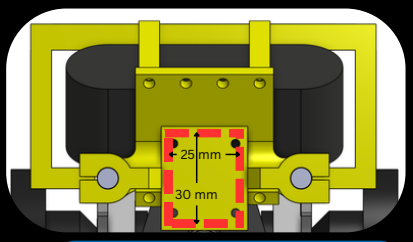


Figure 2.7 - IMU Slot

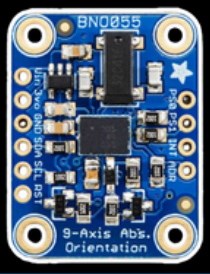


Figure 2.8 - MPU6050

Figure 2.9 - BNO055

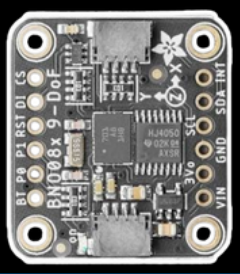


Figure 2.10 - BNO085

IMU Option	Gyro Sensitivity Error (Datasheet)	Gyro Sensitivity Error (Testing)	Size - L x B (mm)	Verdict
MPU6050	±3%	±3.78% (90-86.6)/90	21 x 16	Too inaccurate
BNO055	±1%	±1.67% (90-88.5)/90	40 x 20	Too large
BNO085	Not Specified	±0.89% (90-89.2)/90	25.6 x 22.7	**High accuracy and compact size**

Note: Error calculated while testing was calculated over 25 trials (taken average), (to 3 s.f). For 90° rotation, IMU was mounted on DS3235 Servo.

Points Covered: Sensor placement justification (Section 2), Engineering reasoning for decisions (Section 4), Risk identification and mitigation (Section 4)

SENSOR PLACEMENT RATIONALE

HikVision Webcam

Mounted at a downward angle right at the top of the robot (lens at 285mm) for a wider detection range. Having such a high mount minimizes dead spots near the robot while still having a large, long field of view. We also set the exposure manually to reduce flickering due to the height.

Adafruit BNO085 IMU

Mounted above the camera, just below the permitted height limit. Since the motor uses magnetic force, it interferes with the IMU's accelerometer. Therefore, we have placed it as far from the motor as possible to minimize magnetic interference, and the shaft holding it is doesn't touch the motor, which minimizes vibratory interference.

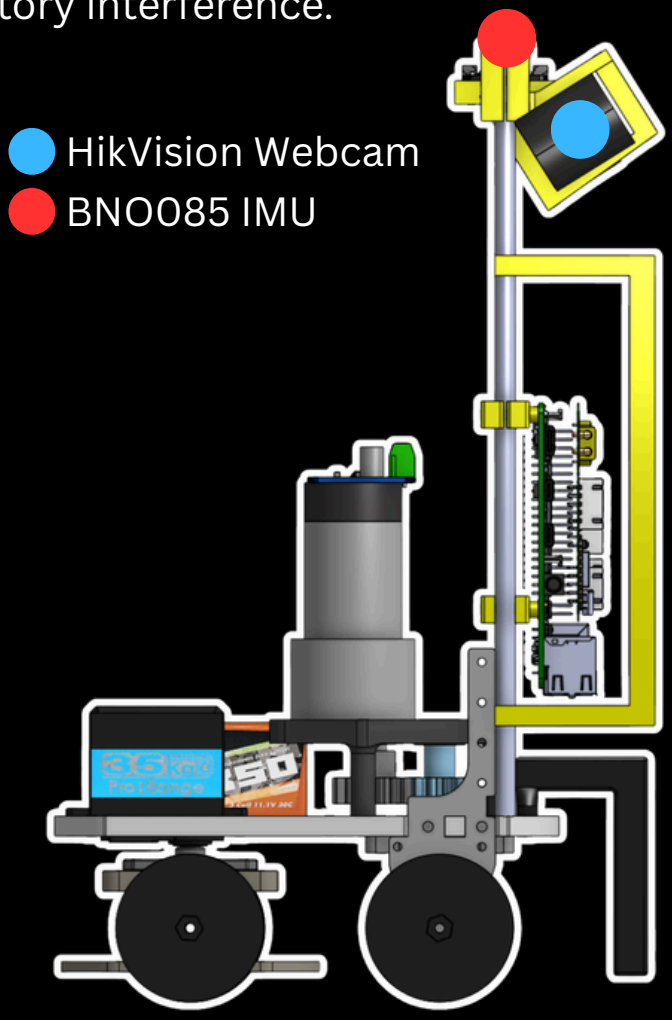


Figure 2.11 - Sensor Placement Diagram

COLOUR CALIBRATION STRATEGY

Stage 1: Record

- We created a python script to record everything in our camera's field of view for a set period of time.
- To get a proper recording for calibration, we would manually move the robot, capturing all the game elements.
- LEDs on the back of the robot show us whether the robot is recording or not (red LED shows not recording, green LED shows recording in progress).

Stage 2: Calibrate

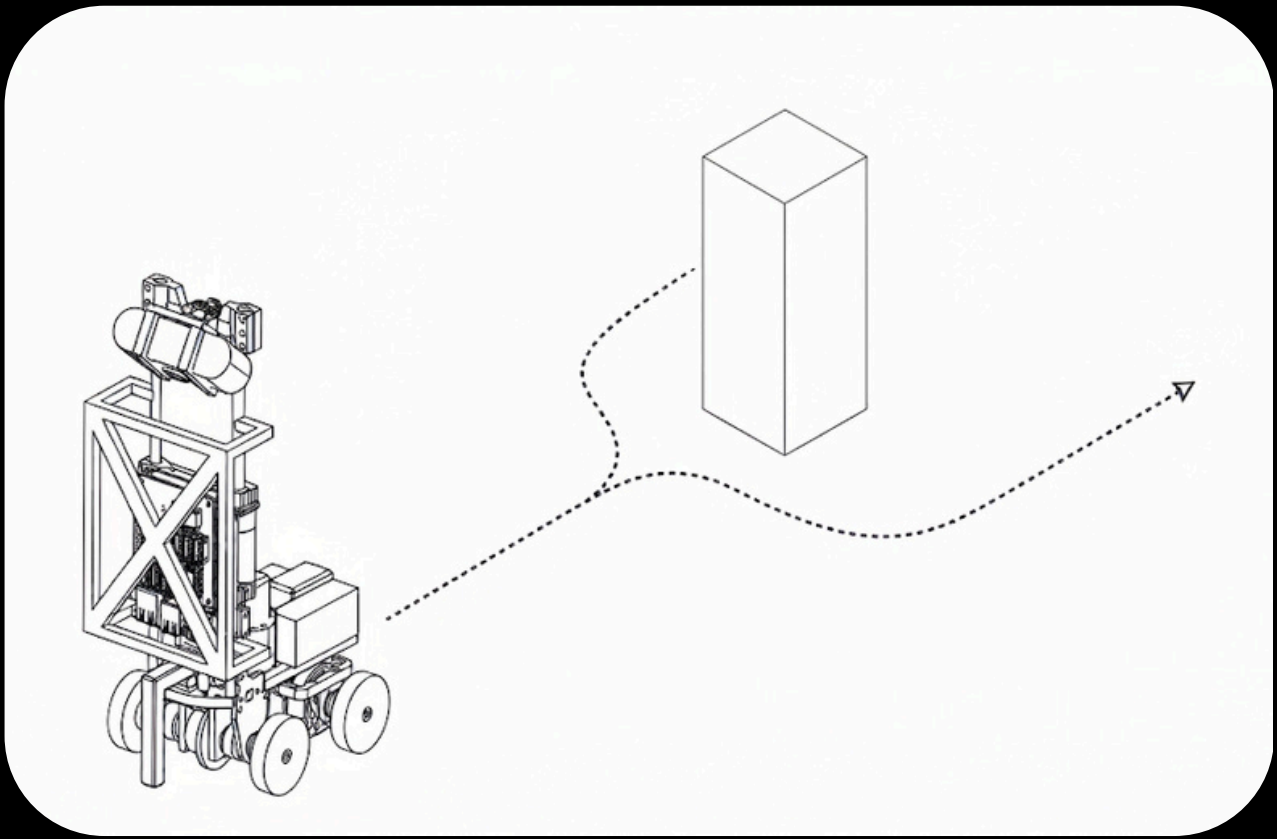
- We created a Matlab script allowing us to import a video, navigate to any timestamp, and calibrate any colour by clicking on a specific frame with that colour, and then alter its HSV values using sliders.
- However, the Matlab script was heavy to run, and sometimes faced lots of lag on our laptop.
- Therefore, to mitigate risk of failed calibration during the competition, we decided to also make a Python script run on our Raspberry Pi for the same purpose (which directly pulls the latest video from the "videos" folder).

Stage 3: Verify Calibration

- After calibrating, we can play the entire video.
- This will overlay the areas which match each colour.
- This allows us to check if the OpenCV is detecting the blocks as their respective colours consistently.

SECTION 3

Software Architecture & Obstacle Strategy



Points Covered:

- Code modularity
- State machines / flow logic
- Lane following & obstacle obedience strategy
- Explanation of algorithms
- Code documentation

Points Covered: - Code modularity (Section 3), Flow logic (Section 3)

SOFTWARE OVERVIEW

- Our programming is split into 2 languages - Python and C++
- Python is the main programming language used, to code the Raspberry Pi. This is the primary software, as it handles drive, steering, and vision, and takes input from the ESP32 for encoder and heading.
- We use C++ to code the ESP32-C3, which takes input from the IMU and encoder.
- The python code on the Pi uses multiprocessing on 3 of the CPU cores.
- All code is available on our Github repository.

Raspberry Pi Process Architecture

- We use multiprocessing on the Pi, which allows us to run our 3 processes - drive, vision, and encoder/IMU - simultaneously
- This decreases the loop time, makes the robot more versatile and makes the code much more modular and easier to maintain

Main Process
Spawns all processes and exits

CameraProcess

- OpenCV HSV pipeline
- Uses the vision_pipeline code for input
- Controls the camera, detects obstacles, walls, lines within the ROIs, and saves the data in multiprocessing variables

DriveProcess

- Main decision-making loop
- Controls the servo & DC motor, takes input from all processes, and contains the main logic
- Decides obstacle avoidance, parking, and PID steering based on IMU, encoder, and camera data

IMUandEncoder

- Communication with ESP32-C3 over USB (UART)
- Sends basic commands and recieves raw IMU and encoder data
- Formats and stores the data in multiprocessing variables for the DriveProcess to use.

Points Covered: - State machines (Section 3), Code modularity (Section 3), Flow logic (Section 3)

OPEN CHALLENGE STATE MACHINE

- Our open challenge code operates using 3 main principles - wall PID, IMU reading PID, and turn logic.
- It uses only 2 sensors - the camera and the IMU's gyroscope.

State Machine

Initalization
Runs all 3 processes
Sets turn counter, IMU, encoder to 0

1st Lane
Sets target heading to 0, runs on IMU PID (difference from 0 is error) until line detected

Direction Decision
Once Line ROI is triggered, the robot sets 'orange_flag' and 'blue_flag' to true/false depending on the colour

While counter <12

Turn
Based on the colour flag, the robot adds 90 to its target heading (mod 360), and takes the turn once the wall ahead enters the 'close_black' region. 1 is added to the turn counter

Straight Lane
The robot runs straight until the line is detected, using IMU correction from target and steering to the middle of the lane if side walls are detected

Last Straight
After the turn, the robot runs straight using only IMU and walls, and stops after 1.4 seconds in the middle of the lane

Points Covered: - Explanation of Algorithms (Section 3), Code modularity (Section 3), Flow logic (Section 3), Risk mitigation (Section 4)

OPEN CODE EXPLANATION...

- As seen on the previous page, the open challenge operates purely on vision and IMU input, without requiring any other sensor data.
- The primary input source is the HikVision webcam, which uses custom OpenCV Regions of Interest (ROIs) to detect the blue/orange lines, front walls, and side walls.
- The IMU data is constantly sent to the Pi, and is used for heading correction and turns. We use PD, with the lane angle setpoint as the target and deviation from that as the error, to keep the robot from crashing into the walls
- A turn counter tracks each turn, and after 12 turns, the robot uses 2 conditions - the 'far_wall' ROI and a timeout - to stop precisely.
- We have used 2 conditions as sometimes the camera detection from a far distance was faulty, so the timeout acts like a safety net. This mitigated a large risk, as sometimes the robot would just keep going if the wall wasn't detected.
- This proved more reliable than encoder counts or distance sensors.

Primary Algorithms Used:

1. **IMU/Heading PID:**
   - Used to make sure that the robot is in the center of the lane whenever it is running. It takes input from the IMU and outputs to the servo. We also use this while turning, by adding 90 to the setpoint (mod 360) to make the robot turn without having to force the servo.
2. **Wall Centroid PID:**
   - Used to push the robot towards the center of the lane when it is close to the walls. It takes input from the camera's side_wall ROIs and outputs with a +/- 2x PID nudge towards the center
3. **Acceleration Ramp:**
   - We implemented an acceleration ramp formula to gradually increase the speed of the robot when starting from rest.

Points Covered: - State machines (Section 3), obstacle obedience strategy (Section 3), Explanation of algorithms (Section 3),

OBSTACLE AVOIDANCE LOGIC

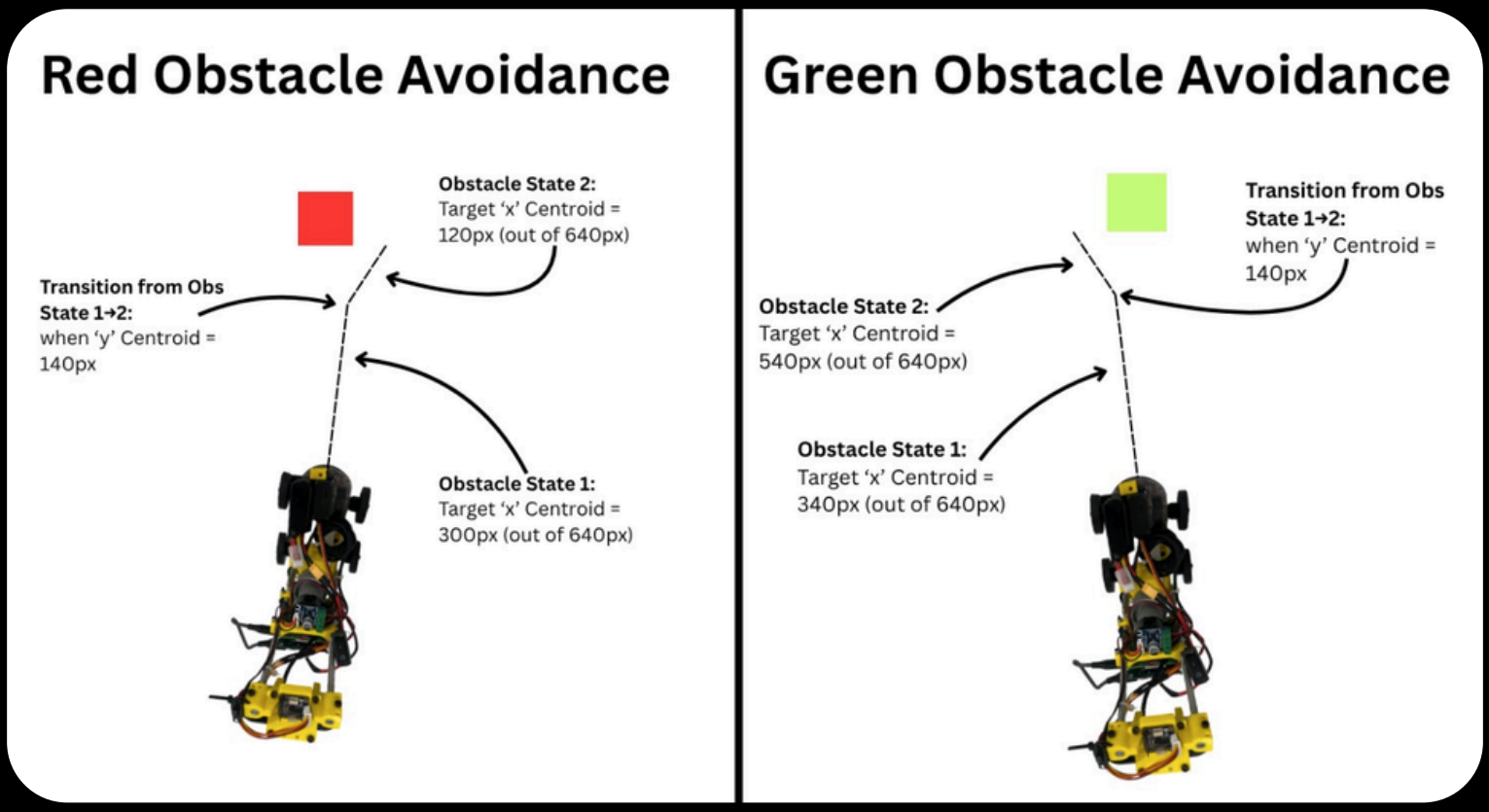


Figure 3.1 - Obstalce Avoidance Diagram

We developed a two-state obstacle avoidance algorithm using HSV colour detection and centroid tracking. The robot identifies red and green obstacles and adjusts its steering based on their position within the camera frame. The states are:

- **State 1 - Following:** The robot extracts the centroid of the nearest obstacle using OpenCV. Next, it uses a PD controller to move till the obstacle's centroid is roughly in the x-axis center of the frame - 270px for red and 370px for green. This lets the robot get close to the obstacle, making sure it doesn't drift out of the frame.

- **State 2 - Avoidance:** Once the obstacle's Y-centroid reaches 140px or enters the close_block ROI, the target X-centroid shifts to 120px for red and 520px for green. The PD controller automatically turns the servo sharply towards the correct side of the obstalce, avoiding it.

Points Covered: - State machines (Section 3), obstacle obedience strategy (Section 3), Explanation of algorithms (Section 3),

OBSTACLE RISK MITIGATION...

While testing, we noticed an error with the obstacle avoidance logic. When an obstacle was detected around a corner (shown in Fig. 3.2a), the robot would try and follow it to the center, but in the process crash into the inner walls. This was a major risk, as due to this the robot would continously hit itself onto the walls and not complete the round.

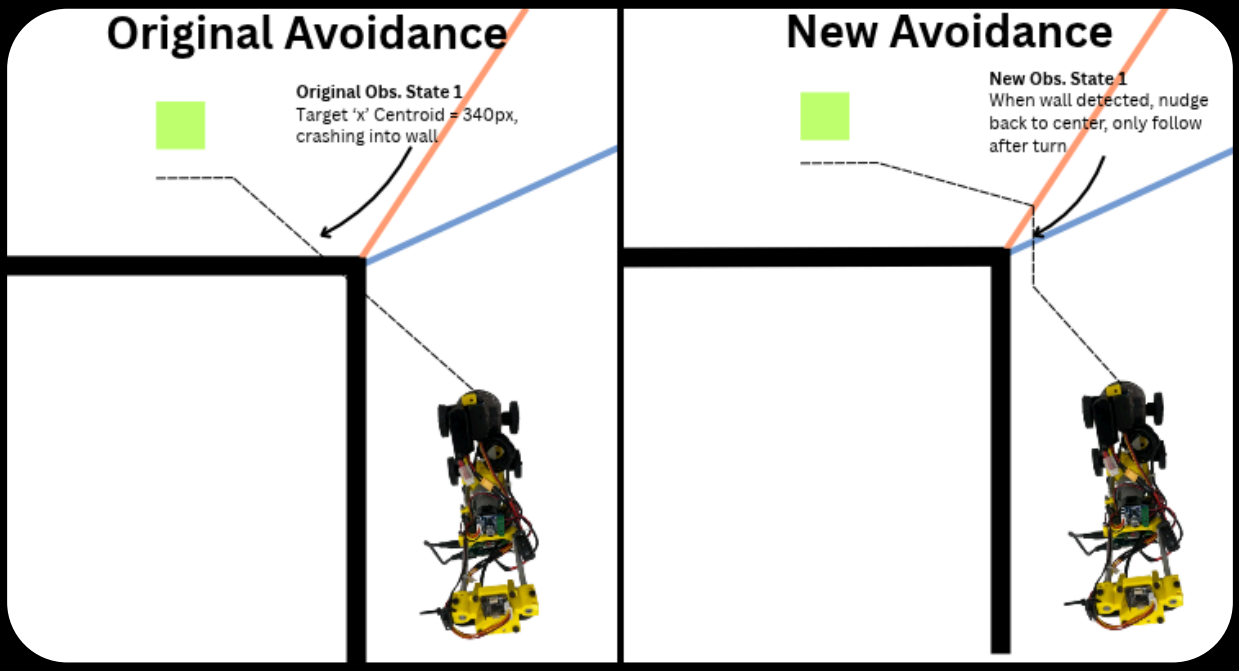


Figure 3.2a - Old Logic

Figure 3.2b - New Logic

To mitigate this risk, we decided to add a condition in obstacle state 1 where if the wall is detected closer than the obstacle, it first navigates away from the wall and then follows the obstacle again. This is applied in 2 of the conditions:

- When blue_flag is true (counterclockwise) and green is detected
- When orange_flag is true (clockwise) and red is detected

This was applied in only these 2 as when there was green in orange flag or red in blue flag, the robot automatically steers away from the wall as a result of the centroid PID correction being in the opposite direction.
As seen in Fig. 3.2b, the robot now steers back to straight when there is a wall between it and the obstacle, and then continues to track the obstacle after the corner is passed

Points Covered: - State machines (Section 3), Code modularity (Section 3), Flow logic (Section 3), Obstacle obedience strategy (Section 3)

OBSTACLE AVOIDANCE ALGORTHM

- The code for obstacle avoidance revolves around using the error as the input, servo as the output, and centroid as the validation condition.
- First, the code extracts the centroid, plots it on the x- and y-axes, and saves it as red_b.value or green_b.value.
- Next, depending on the colour and direction, it chooses the correct value to set for the error, which is inputted into the PD controller
- The PD steering stops once the block has reached the target centroid or is out of frame.
- This is a simplified snippet of the 2-state avoidance code (without the PD control and corner edge case):

```
if green_b.value and (not red_b.value and time.time() - r_time > 0.01) and not lap_finish: # if green block is deteced
    g_time = time.time()
    r_time = 0
    centroid = centr_x.value
    print("default green following")
    if obstacle_state == 1: #green block is far
        print(f"center greeen...")
        err = 370 - centroid #steer till green is roughly x-axis center of frame (640px frame)
    elif obstacle_state == 2: #green block is near
        print(f"left greeen...")
        err = 520 - centroid #steer left of the green block

elif red_b.value and (not green_b.value and time.time() - g_time > 0.01) and not lap_finish: #if red block is detected
    r_time = time.time()
    g_time = 0
    centroid = centr_x_red.value
    print("default red following")
    if obstacle_state == 1: # red block is far
        print(f"center red...")
        err = 270 - centroid #steer till red is roughly x-axis centre of frame (640px frame)
    elif obstacle_state == 2: #red block is near
        print(f"right red...")
        err = 120 - centroid #steer right of the red block
```

Figure 3.3 - Obstacle Avoidance Snippet

Points Covered: - State machines (Section 3), Code modularity (Section 3), Flow logic (Section 3), Obstacle obedience strategy (Section 3)

OBSTACLE STATE MACHINE

- Our obstacle challenge code operates using 5 main principles - wall PID, heading PID, 2-state obstacle avoidance, and parking accuracy.

State Machine

Initalization
Runs all 3 processes, initializes all variables

Direction Decision/Initial Manouver
Camera checks the left and right to see which side is the black wall. Depending on that it sets the direction to orange/blue. Next, it executes a sharp turn to exit the parking space, and then continues in the correct direction

While counter <12

Turn
Based on the colour flag, the robot adds 90 to its target heading (mod 360), and takes the turn. 1 is added to the turn counter

Obstacle Avoidance
Runs obstacle state 1 (following the block) and then obstacle state 2 once the block is close enough. Also uses the edge case of walls between the robot and the block.

IMU/Wall Correction
If no obstacles are seen, use IMU PID (heading) and wall nudges (like in the Open Challenge) to keep the robot centred

Parking
The robot reverses into the wall of the 12th turn and sets encoder counts to 0. Then, it moves ahead, past the pink wall, (using encoder counts) and takes a reverse U-turn while aligning itself into the correct lane. Then, it moves until it is parallel to the parking space and slightly ahead, reverses perpendicularly into the space, and aligns itself with a sharp turn

Points Covered: - State machines (Section 3), Code modularity (Section 3), Flow logic (Section 3), Obstacle obedience strategy (Section 3)

IMU PD CONTROL ALGORITHM

- As mentioned earlier, we use a PD controller for our steering mechanism.
- We chose to use PD instead of PID (by setting the integral term to 0) to avoid windup during block avoidance transitions. We learnt this lesson when every turn around a block was treated as a huge error, making the robot over-correct and hit walls constantly
- The PD controller takes input from the BNO085 IMU's heading, which is sent to the Raspberry Pi through the ESP32-C3.
- It outputs to the servo, adjusted to fit the mechanical limits - we capped it at 30 to 150 degrees, as beyond that the wheels would hit the back of the robot and the turn would be too sharp to execute.
- An innovative feature we have used is a 'multiplier' parameter to scale the aggressiveness of the turn. Instead of having multiple kP and kD values to turn with different sharpnesses, we factored in an input from 1 to 4, which decides how aggressive the turn should be. This makes the code much more modular and easy to change
- We generally use a multiplier of 1 for straight-line holding, 1.5-2 for obstacle avoidance, and 3 for sharp parking turns

```
### CORE PID
error_gyro = heading - setPoint_gyro
if error_gyro > 180: #angle wrap around
    error_gyro -= 360
corr = error_gyro

pTerm = kp * error_gyro * multiplier
dTerm = kd * ((error_gyro - prevErrorGyro) / dt)
totalErrorGyro += error_gyro
iTerm = ki * totalErrorGyro
correction = pTerm + iTerm + dTerm
servo.setAngle(95-correction) #95 is the mechanical centere
```

Figure 3.4 - Heading PID code

Points Covered: - Explanation of algorithms (Section 3), Calibration (Section 2), Obstacle obedience strategy (Section 3)

HSV PIPELINE SYSTEM

- We have an extremely robust and advanced camera pipeline system, as that is the main source of input for both obstacle and open challenge.
- This pipeline operates using HSV - hue, saturation, and value - to detect the colours of the objects. We chose HSV over RGB as it can isolate the colour/hue from the brightness, allowing the detections to be less shaky and more solid in different lighting.
- We use OpenCV to draw ROIs - Regions of Interest - around the camera feed, which detect only certain colours and certain sizes. This helps to isolate the correct detections and make it more error-proof.
- We have separate pipelines for open and obstacle challenges, to minimize the number of overlapping ROIs and make the open challenge loop time faster (no red/blue/pink detection).

Detection Logic

- Within the ROIs, the pipeline isolates the colour (based on the calibration) an draws a contour around it. It also extracts the centroid, allowing us to track it accurately. All of this is done using OpenCV.
- If there are multiple red/green objects detected, it only feeds the largest and closest (low y-coordinate) one to the main code. This prevents the robot from trying to go in 2 directions at once
- The pink colour has priority over everything else, so the first step (except in parking) is to move away from the pink centroid

Why HSV over ML?

- We have chosen to use an HSV pipeline over AI/Machine Learning models as it has a much higher frame rate and a lower latency
- In addition, HSV can be calibrated extremely fast, while ML requires a huge database of photos. Taking all these photos in the lighting at a competition would consume a lot of time

CODE DOCUMENTATION

- All our code is located at our GitHub repository. It can be accessed by scanning the QR code on pg. 2 or following this link: https://github.com/Zkwadia/WRO_FE_Team_Glitch
- All code files are under /src/, which is divided into OpenCV (for calibration), ESP (ESP32 codes), and Pi (all Rpi Codes).
- To run these codes on your own Raspberry Pi, you will have to edit the paths (as there are some local files) and create your own services accodring to your specifications. You can use our service codes (under /service_files/ in our repo) for reference
- We also have test codes for each component, allowing you to make sure everything is working before running the full code

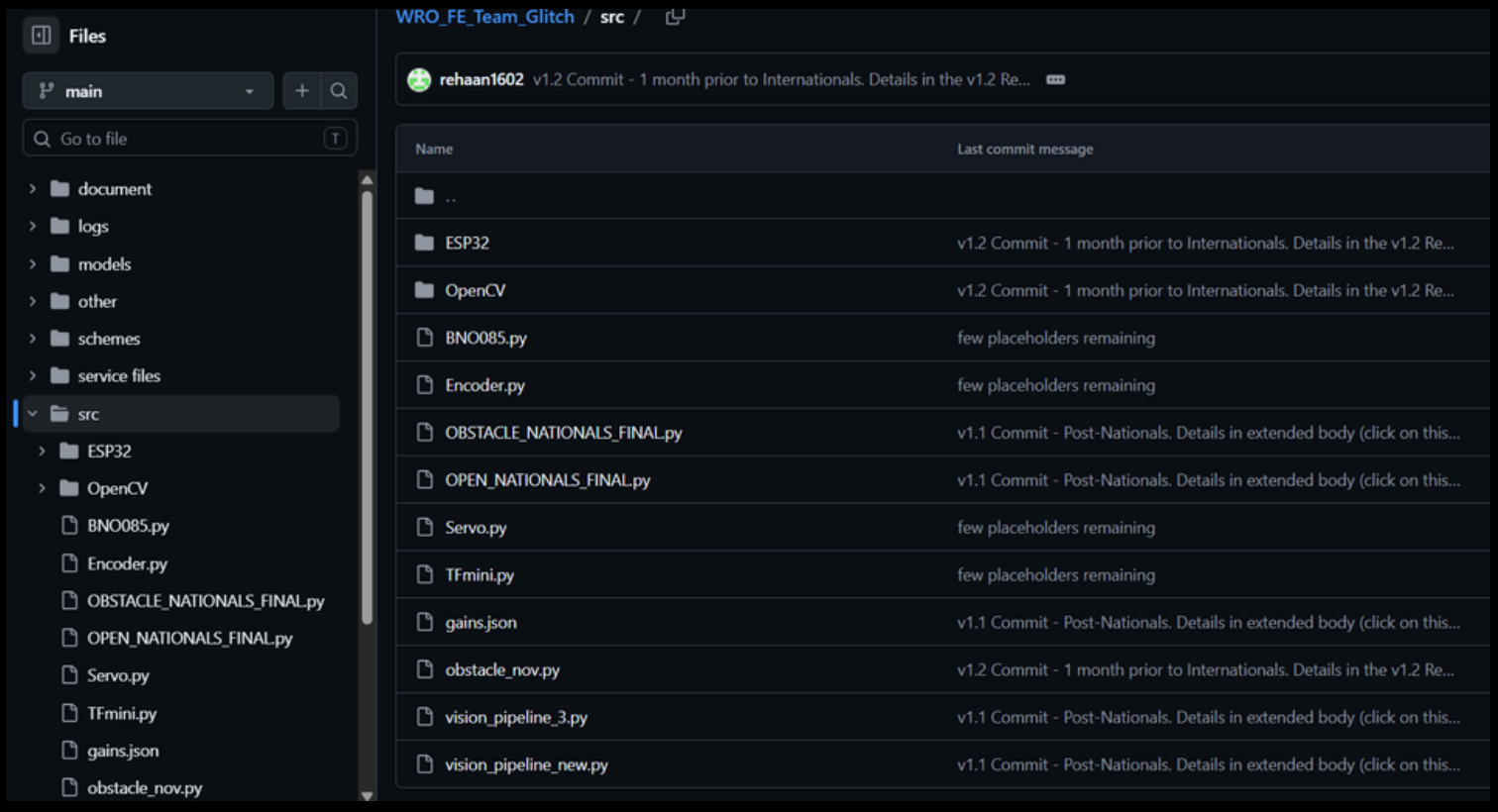
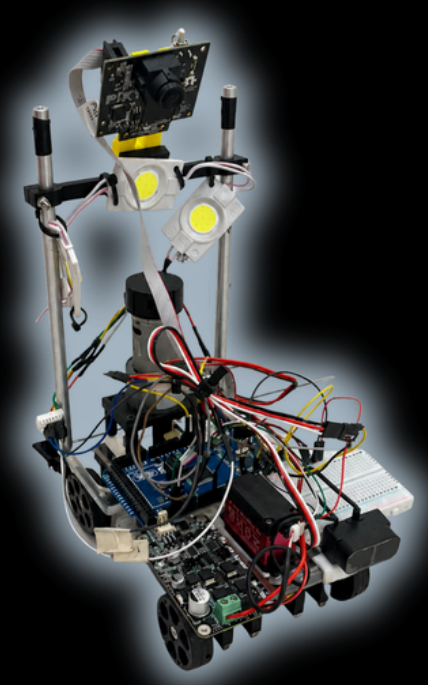


Figure 3.5 - GIt Code Structure

SECTION 4

Systems Thinking and Engineering Decisions

V1



V2

Points Covered:

- How subsystems work together
- Engineering reasoning for decisions
- Constraints & trade-offs
- Iteration and testing cycles
- Risk identification & mitigation

INITIAL ROBOT ELECTRONICS...

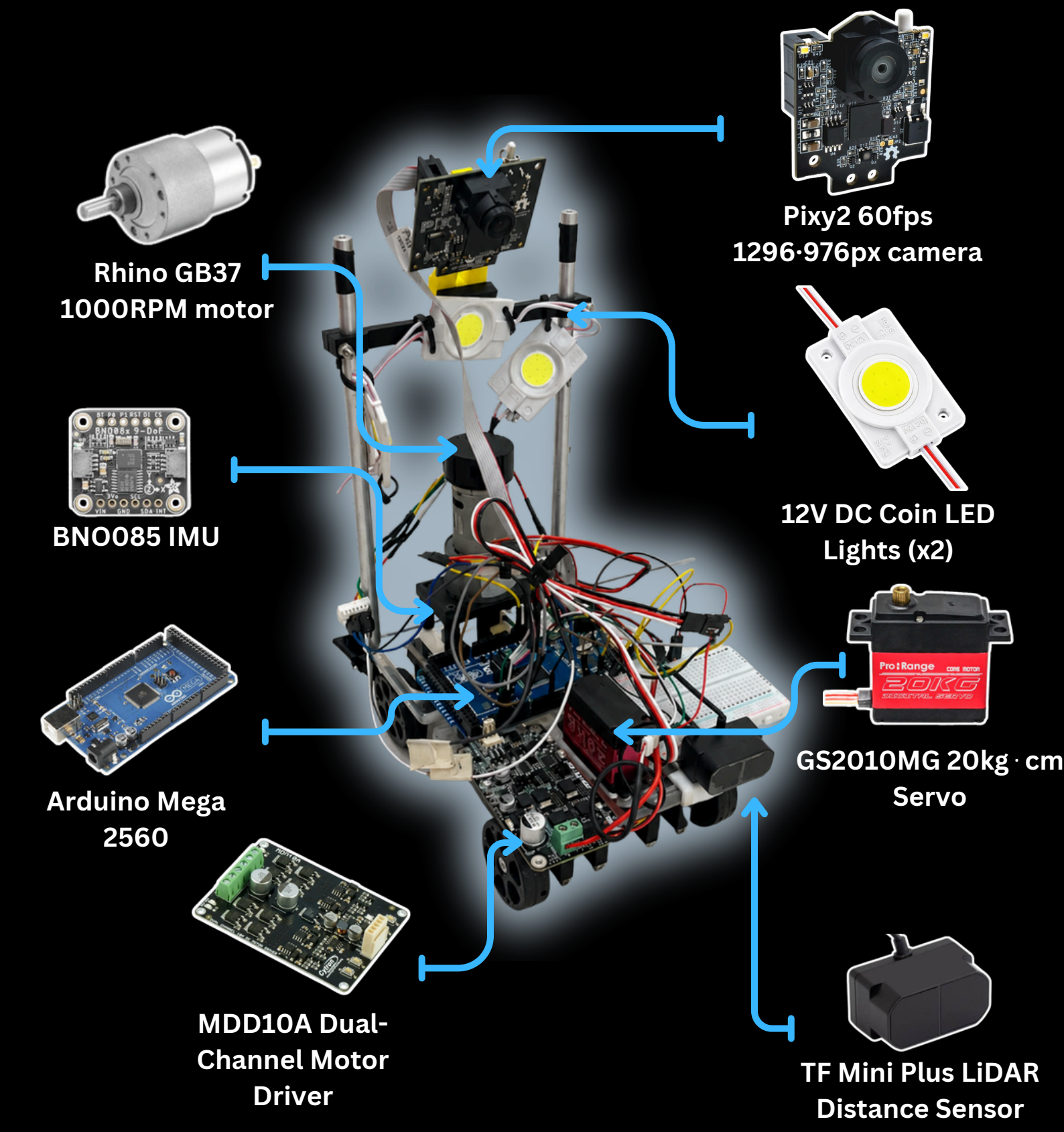


Figure 4.1 - Old Robot Electronics (Annotations)

FINAL ROBOT ELECTRONICS...

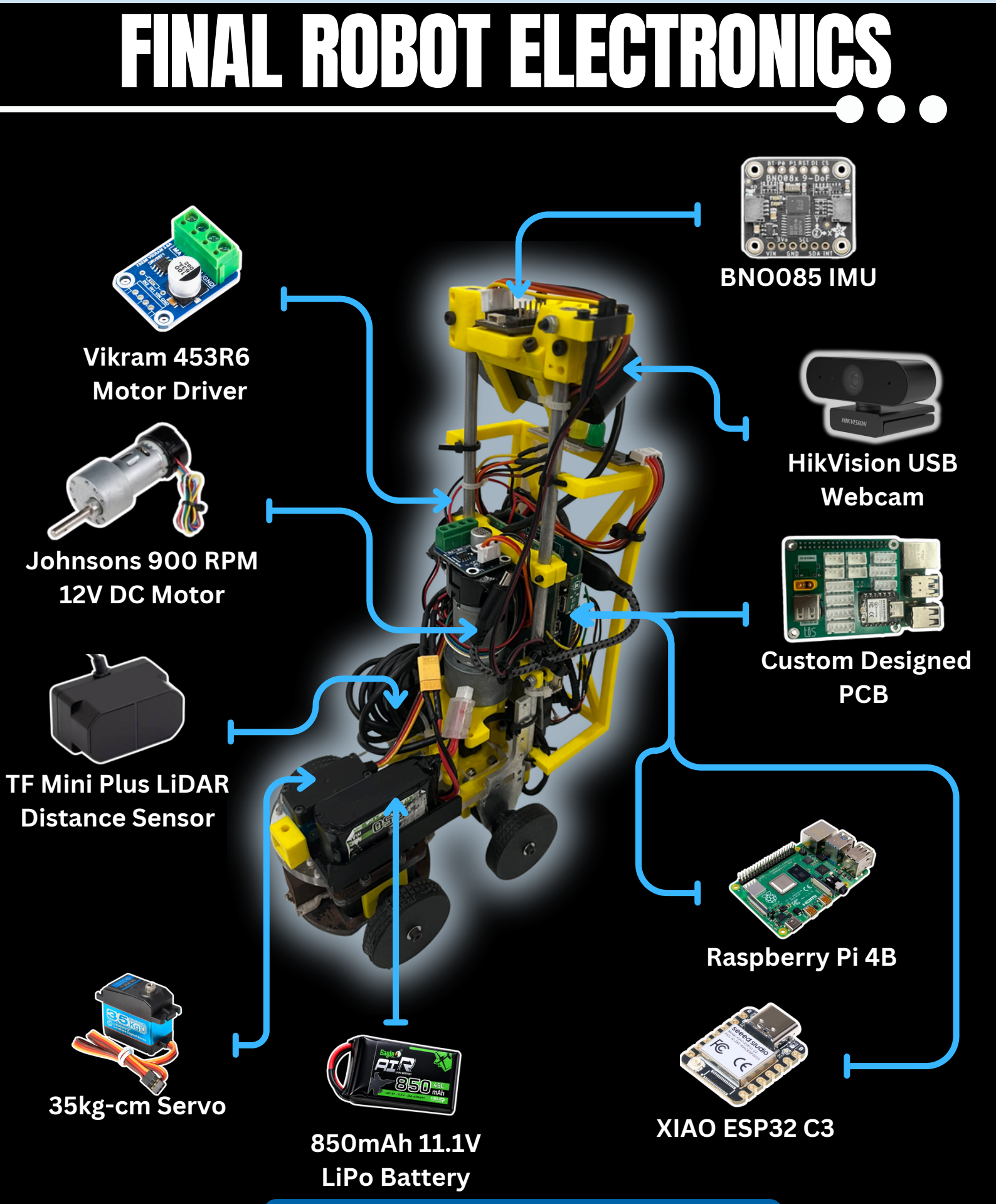


Figure 4.2 - Final Robot Electronics (Annotations)

Points Covered: Iteration and testing cycles (Section 4), Engineering reasoning for decisions (Section 4), Constraints and trade-offs (Section 4), Risk identification and mitigation (Section 4), Sensor selection and placement justification (Section 2).

COMPARING ELECTRONICS...

Version 1 v/s Final Robot

Primary Processor: Arduino Mega 2560 (V1) vs Raspberry Pi 4B (V2)

- **Number of Cores:** Raspberry Pi 4B's quad-core cortex allows it to run 4 processes at once, whereas the Arduno Mega can only run 1.
- **Hardware Specifications:** The Raspberry Pi also has superior RAM (4GB > 8KB), Storage (Raspberry Pi also external MicroSD card) and clock speed (1.5GHz > 16MHz)
- **Camera Compatibility:** The Raspberry Pi also allows us to use a USB webcam via USB 2.0 or USB 3.0. It also is compatible with OpenCV.

Secondary Processor: None (V1) XIAO ESP32C3 (V2)

- We decided to use a secondary processor (XIAO ESP32C3).
- **Reducing Electrical Noise:** Using a secondary processor for the IMU and encoder readings helped us reduce electrical noise (from the motor), stabilising their readings.

Motor Driver: MDD10A (V1) vs Vikram 453R6 (V2)

- **Overcurrent Protection:** After the MDD10A faced 2 failures (due to not having overcurrent protection), we decided to switch to the Vikram 453R6 motor driver which adds overcurrent protection, thermal shutdown and auto-recovery.
- **Higher Voltage Ceiling:** The 453R6 can handle up to 45V DC input, making it capable of handling inductive spikes. This is much safer that the MDD10A's 30V DC ceiling.

DC Motor: Rhino GB37 1000 RPM (V1) vs Johnsons 900 RPM (V2)

- **Torque:** With only 0.17 kgcm of torque, the Rhino GB37 (used in our version 1 robot) was inconsistent.
- We therefore switched to the Johnson's 900 RPM DC Motor with 1.2 kgcm rated torque.

Points Covered: Iteration and testing cycles (Section 4), Engineering reasoning for decisions (Section 4), Constraints and trade-offs (Section 4), Risk identification and mitigation (Section 4), Sensor selection and placement justification (Section 2).

COMPARING ELECTRONICS...

Version 1 v/s Final Robot

Servo Motor: GS2010MG Servo (V1) vs DS3235 Servo (V2)

- **Mitigating Risks:** The GS2010MG used in our first robot faced no issues, however noticing that the DS3235 has more torque, we switched to mitigate risks (especially considering that we planned to add a deadweight, making the parallel steering linkage heavier).

Primary Camera: Pixy2 Camera (V1) vs Hikvision USB DS-UO2 Camera (V2)

- **Processor Compatibility:** At first we used the Pixy2 camera as it was compatible with our Arduino Mega, however after switching to a Raspberry Pi 4B, we switched to the Hikvision DS-UO2.
- **OpenCV:** Additionally, the Hikvision DS-UO2 has OpenCV compatibility, while the Pixy2 didn't.
- **Resolution:** The Hikvision DS-UO2 also has higher resolution (920 x 1080)

Flash LEDs: 12V DC Coin LED (V1) vs None (V2)

- **Earlier Trials:** At first, we tried to use these LEDs to allow our robot to work consistently in uneven lighting.
- **Direct Lighting:** However, with such direct lighting, blocks were often confused with white. Therefore, we removed these LEDs.

INITIAL ROBOT HARDWARE...

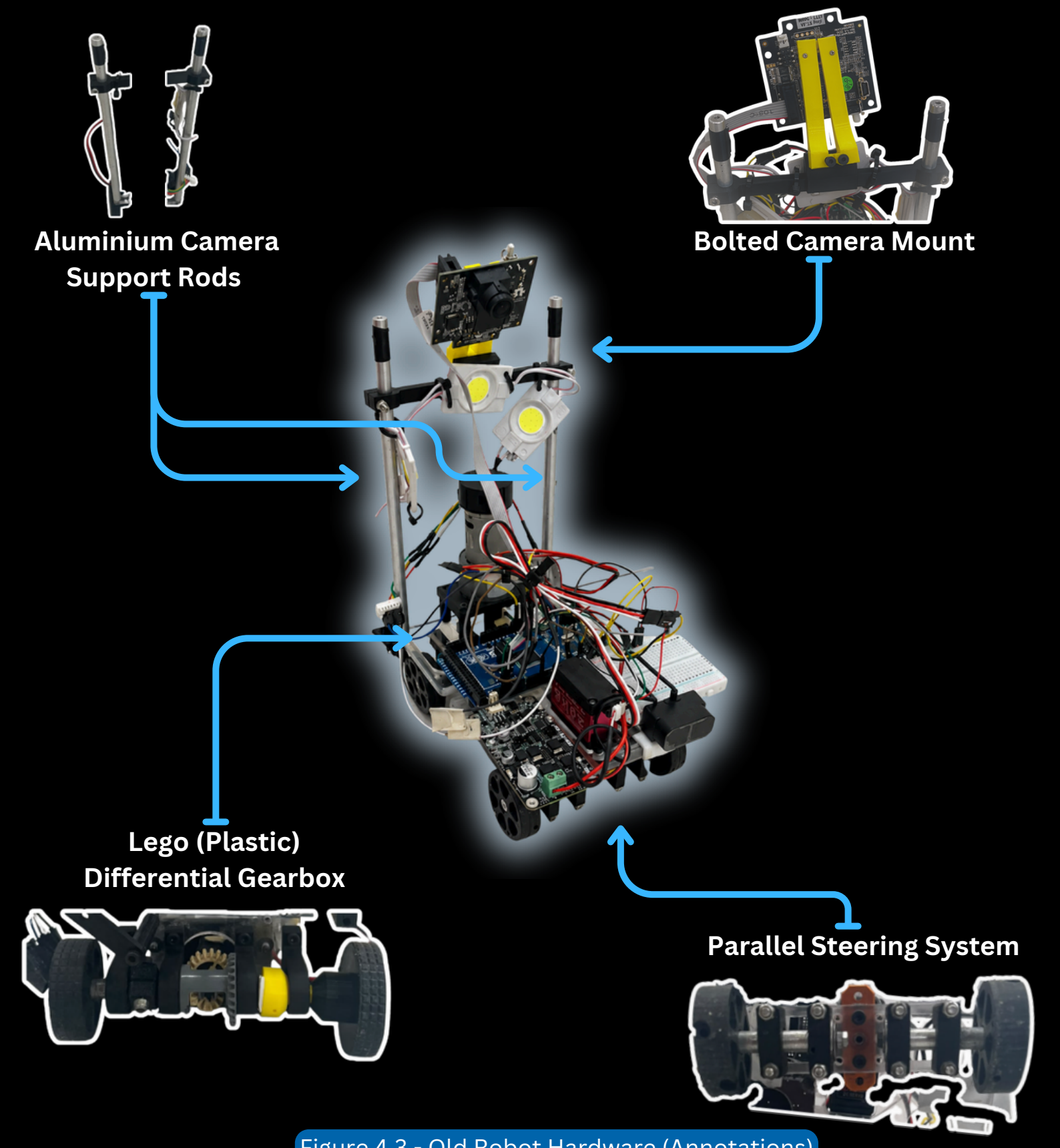


Figure 4.3 - Old Robot Hardware (Annotations)

FINAL ROBOT HARDWARE...

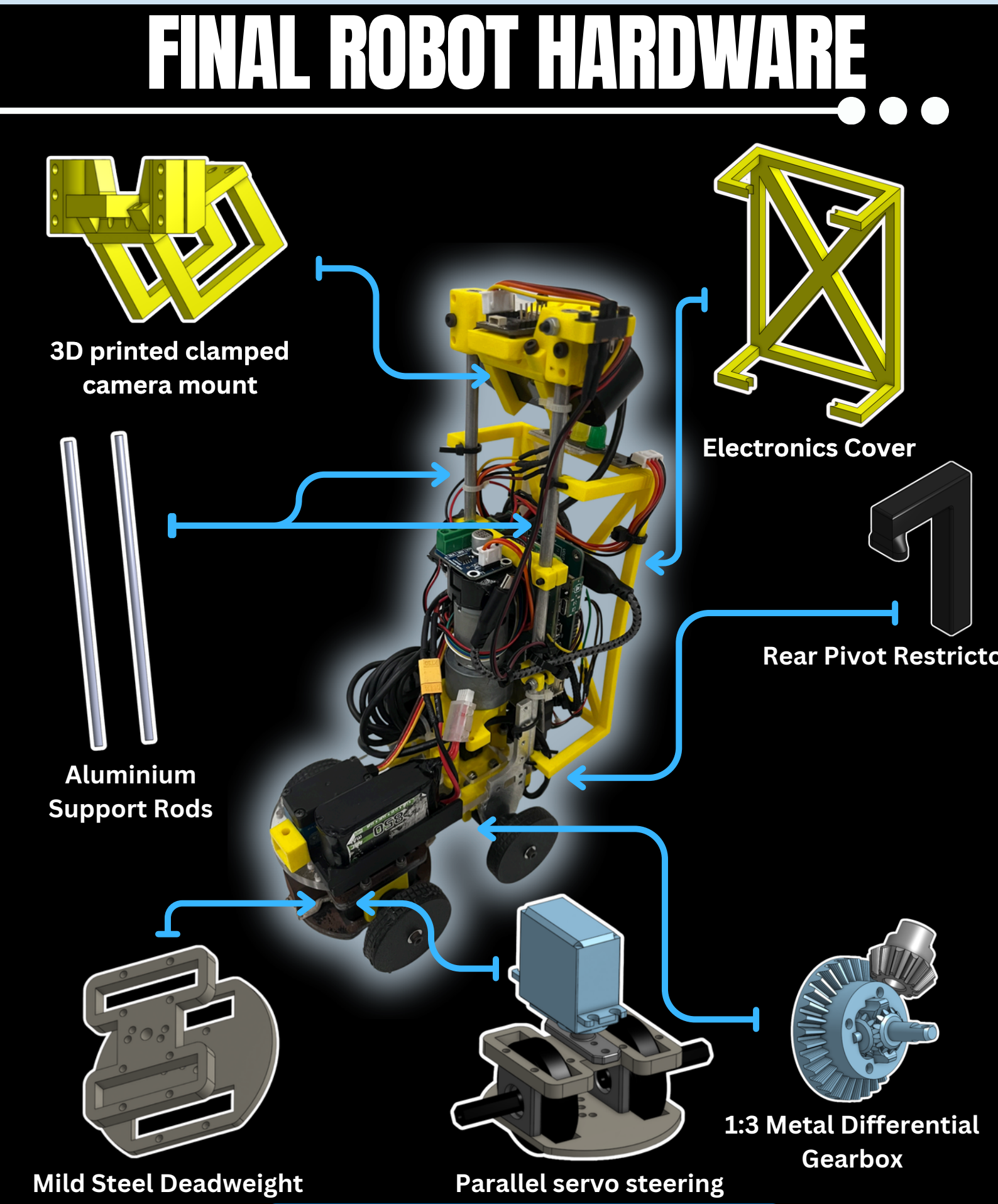


Figure 4.4 - Final Robot Electronics (Annotations)

Points Covered: Iteration and testing cycles (Section 4), Engineering reasoning for decisions (Section 4), Constraints and trade-offs (Section 4), Risk identification and mitigation (Section 4), Chassis design choices (Section 1), Mechanical stability (Section 1)

COMPARING HARDWARE...

Version 1 v/s Final Robot

Electronics Mounting: Basic Electronics Mounting (V1) vs Custom 3D-Printed Mounts (V2)

- **Space Optimisation:** Our initial electronics arrangement occupied more space and made component placement less organised. We designed custom 3D-printed mounts to secure the PCB and other electronics directly onto the aluminium support rods.
- **Reliability:** The new mounts keep components fixed during movement, reducing vibration and the risk of loose connections. This also makes wiring more organised and maintenance easier.

Camera Mounting: Bolted Camera Mount (V1) vs 3D-Printed Clamped Mount (V2)

- **Stability:** Our initial camera used a bolted mounting arrangement. We replaced it with a custom 3D-printed clamped mount that holds the camera securely on the support rods.
- **Design Flexibility:** 3D printing allowed us to design the mount around the camera's dimensions and intended position, giving us a more compact and integrated structure.

Mechanical Stability: Initial Chassis (V1) vs Deadweight and Rear Pivot Restrictor (V2)

- **Risk Mitigation:** Our final robot includes a front mild-steel deadweight and a rear pivot restrictor to reduce the risk of the robot tipping backwards, particularly during sudden acceleration or sharp turns.
- **Trade-Off:** These components increase the robot's mass, but we accepted this drawback to improve stability and protect the robot during runs.

Points Covered: Iteration and testing cycles (Section 4), Engineering reasoning for decisions (Section 4), Constraints and trade-offs (Section 4), Risk identification and mitigation (Section 4), Chassis design choices (Section 1), Mechanical stability (Section 1)

COMPARING HARDWARE...

Version 1 v/s Final Robot

Chassis Design: Wider Chassis (V1) vs Slimmer Polycarbonate Chassis (V2)

- **Slimmer Chassis:** Our initial chassis had a wider layout, taking up unnecessary space. We redesigned it to be slimmer and more compact, reducing excess material and making the robot easier to manoeuvre around obstacles.
- **Weight Reduction:** The new design reduces unnecessary structural mass, lowering the load on our drive motor and allowing us to prioritise speed while maintaining stability.

Differential Gearbox: LEGO Plastic Differential (V1) vs 1:3 Metal Differential (V2)

- **Durability:** Our initial robot used a LEGO plastic differential gearbox. However, we switched to a metal differential for a more rigid and durable drivetrain that could withstand repeated high-speed runs.
- **Reliability:** The metal differential provides a stronger connection between the motor and rear wheels, while still allowing both wheels to rotate at different speeds during turns. This reduces tyre scrub and improves consistency.

IMU HEADING WRAP AROUND

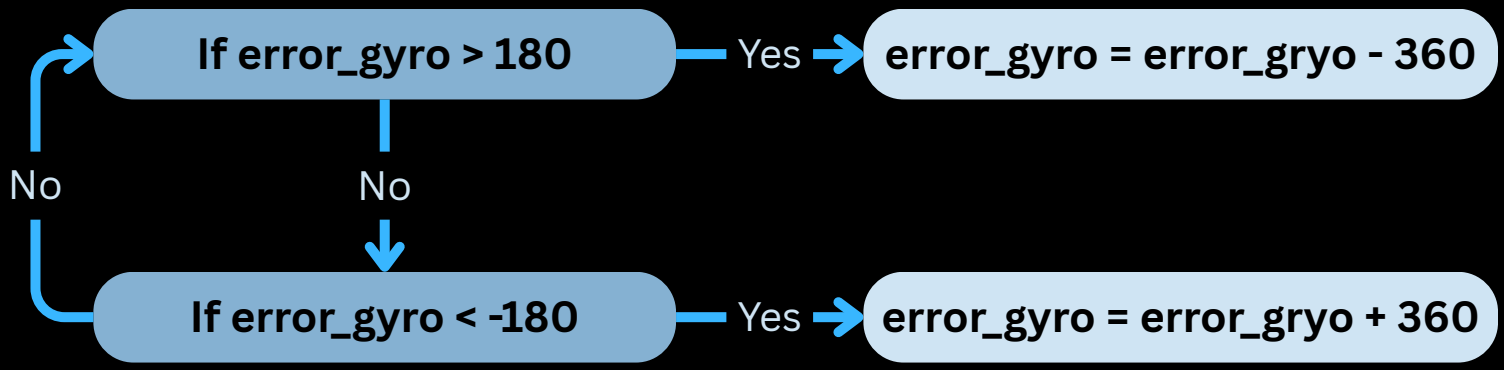
The Problem:

- Heading is circular, so it jumps at the boundary - crossing from +179° to −179° is a 2° turn that naive subtraction reports as −358°.
- The PID cannot tell the difference - it sees a huge error and commands a full-lock turn in the wrong direction, usually mid-corner where recovery room is smallest.

Our Solution:

- Normalize the error, not the heading - wrapping the raw reading just moves the discontinuity; wrapping the difference removes it.
- Handle both directions - subtracting 360 above +180 fixes only half the circle; without the matching +360 branch below −180, the fault survives in one turn direction.

Flow Chart:



Points Covered: How subsystems work together (Section 4)

SUBSYSTEM INTEGRATION...

Mobility & Software

Our drivetrain is controlled using PD steering and speed PID. Live sensor feedback allows the robot to continuously adjust its steering and speed instead of following fixed movements.

Sensor & Software

The camera and IMU work together to provide the software with vision, distance, and heading data. Combining multiple sensors makes navigation and obstacle avoidance more reliable.

Power & Sensors

We separated the motor and logic power rails to reduce electrical noise. This keeps sensor readings stable, even during rapid acceleration and steering.

Frame & Sensors

The chassis was designed around sensor placement. The elevated camera improves visibility, while the IMU is positioned to maximise accuracy and minimise interference.

Frame & Mobility

Our lightweight, rigid chassis improves traction and stability while reducing load on the drivetrain. This allows faster, smoother, and more consistent movement.

Overall System

Each subsystem supports the others. The frame improves sensor performance, sensors guide the software, the software controls mobility, and the power system keeps everything running reliably, creating a robot that can adapt dynamically with minimal hard-coding.

Points Covered: Risk identification and mitigation (Section 4), Engineering reasoning for decisions (Section 4)

RISK MITIGATION

- We identified the following risks and mitigated them.

Risk Identified	Mitigating Risk
Robot tipping during high acceleration.	Added dead weight to lower the centre of gravity and widened the wheelbase to improve stability and prevent tipping. We also added a rear pivot restrictor.
Wiring failures	Designed and manufactured a custom PCB to make the wiring more secure, organized, and reliable.
Limited space for electronics	Replaced the MDD3A motor driver with the more compact 7Semi Vikram-453R6, freeing up space for other components.
Motor's electromagnetic signals affected IMU readings	Moved IMU far from the motor (above the camera mount), reducing the impacts of the electromagnetic interference. Additionally, created a dual-rail power system, separating all the electronics from the motor rail.
Robot sometimes hit walls when prioritising obstacle following over wall avoidance	Added a condition in obstacle state 1 where if the wall is detected closer than the obstacle, it first navigates away from the wall and then follows the obstacle again.

Points Covered: Risk identification and mitigation (Section 4), Engineering reasoning for decisions (Section 4), Code modularity (Section 3), Constraints and trade-offs (Section 3)

ENGINEERING DECISIONS...

Python Threading vs Multiprocessing

The Problem

- When everything was on a single thread/loop, it held camera, HSV, IMU, encoder, steering, but all in sequence instead of parallel
- Blocking reads cost 10–15 ms/cycle, capping the loop at ~65–100 Hz before any logic ran.
- Sense-to-steer lag left the robot committing to stale data, and a busy frame stretched it further mid-run. Sometimes, the robot would hit an obstacle before it detected it due to the slow loops

The Solution

- We used multiprocessing instead of threading, utilizing 3 of the Raspberry Pi's 4 cores.
- The 3 processes - vision, IMU/encoder, and drive - all run in parallel and share the same multiprocessing variables.
- Loop rate is now decoupled from frame arrival - a slow frame reuses the last vision result instead of stalling steering.

Trade-Offs

- Not value corruption - multiprocessing.value already locks single accesses.
- Inconsistency across variables - head.value and dist.value read separately, updated in between, giving a state the robot was never in.

Points Covered: Risk identification and mitigation (Section 4), Engineering reasoning for decisions (Section 4), Code modularity (Section 3), Constraints and trade-offs (Section 3)

ENGINEERING DECISIONS...

Single vs Multi-Stage Parking

The Problem

- The rules expect the robot to be fully inside the parking lot, and parallel to the outer wall
- Because of this, a simple forward drive into the slot fails, as it parks the robot at an angle instead of parallelly
- In addition, the camera is not reliable enough on its own to find the exact point where the robot is parallel to the walls

The Solution

- We developed a multi-stage parking system for the robot:
  - First, it reverses into the 12th turn wall, and resets the encoder counts
  - Second, it moves forward a fixed number of counts (9500/6450 for orange/blue)
  - Third, it takes a u-turn and moves forward until it is right next to the walls and parallel to the parking slot
  - Fourth, it reverses into the slot, perpendicular to the side wall
  - Fifth, it takes a sharp right turn (multiplier as 2.5 or 3 depending on the closeness to the wall) and backs up if needed, completing the parallel parking.

Trade-Offs

- This parking system takes around 21 seconds to execute, which increases our entire obstacle timing. However, this increases the chance of having a full point score.
- While executing the multi-stage parking, the robot sometimes disregards obstacles while following the pink wall. However, this is extremely infrequent, and it is much better than only scoring 7 out of 15 for parking.

Points Covered: Risk identification and mitigation (Section 4), Engineering reasoning for decisions (Section 4), Code modularity (Section 3), Constraints and trade-offs (Section 3)

ENGINEERING DECISIONS...

Arduino Mega vs Raspberry Pi 4B

Our First Solution

- **Arduino Mega:** Previously used as primary processor for all processes.
- **Single Core:** Only has 1 core, forcing all processes to be run in one loop (leading to very high loop time).
- **Poor Hardware (compared to Raspberry Pi):** Low RAM, storage and clock speed further increasing loop times.
- **OpenCV Incompatibility:** Not compatible with OpenCV, limiting camera options to ones with built-in colour recognition software (such as Pixy). Also not compatible with USB webcams.

Our New Solution

- **Raspberry Pi 4B:** Replaced Arduino Mega with Raspberry Pi 4B.
- **4 Cores:** Quad-core processor allows multiple separate processes to run at once (multiprocessing).
- **Better Hardware:** Higher RAM and clock speed. Supports external microSD storage card.
- **Camera Compatibility:** Has USB webcam compatibility (via USB 2.0 or 3.0). Compatible with OpenCV. This allowed us to use HikVision Webcam.

Feature	Arduino Mega	Raspberry Pi 4B
Core Processor	Single Core ATmega 2560	BCM2711 Quad-core Cortex
Clock Speed	16 MHz	1.5 GHz
RAM	8 KB	4 GB
Storage	256 KB Flash Memory	Compatible with external microSD card.

Adding a Secondary Processor

- We added an ESP32C3 XIAO to read encoder and IMU inputs.
- This allowed us to keep these sensors on a separate rail from the motor, making their readings more reliable and consistent.

SCORE PROGRESS BREAKDOWN

Before vs After India Nationals

Before India Nationals

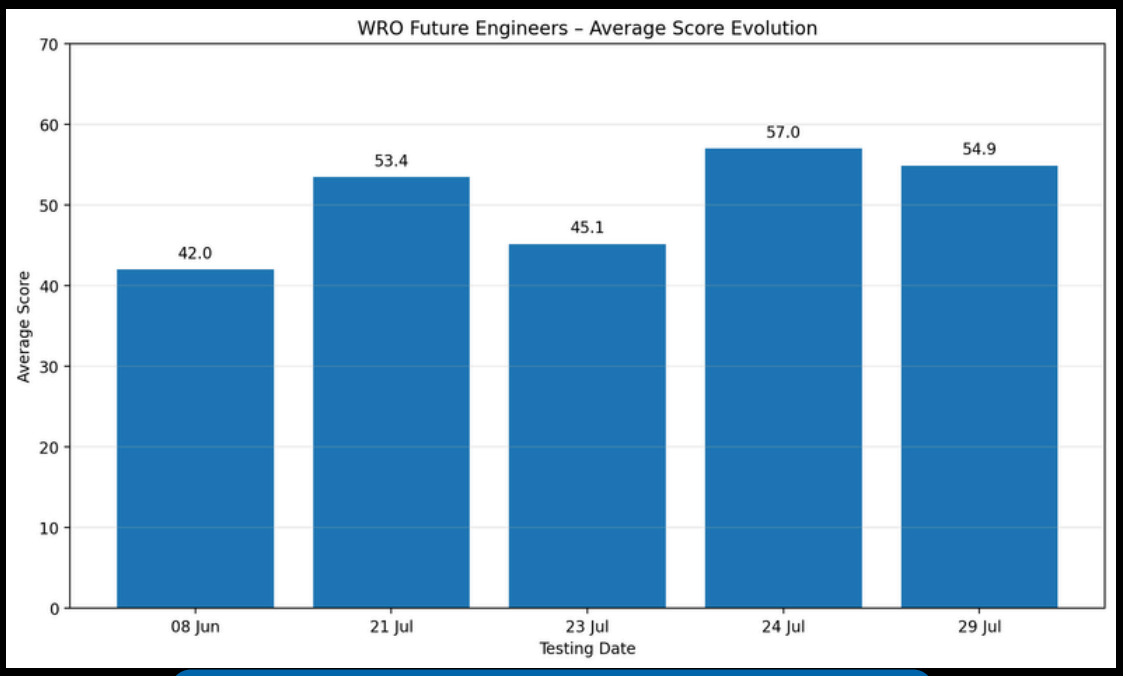


Figure 4.5 - Graph of Scores Before Nationals

After India Nationals

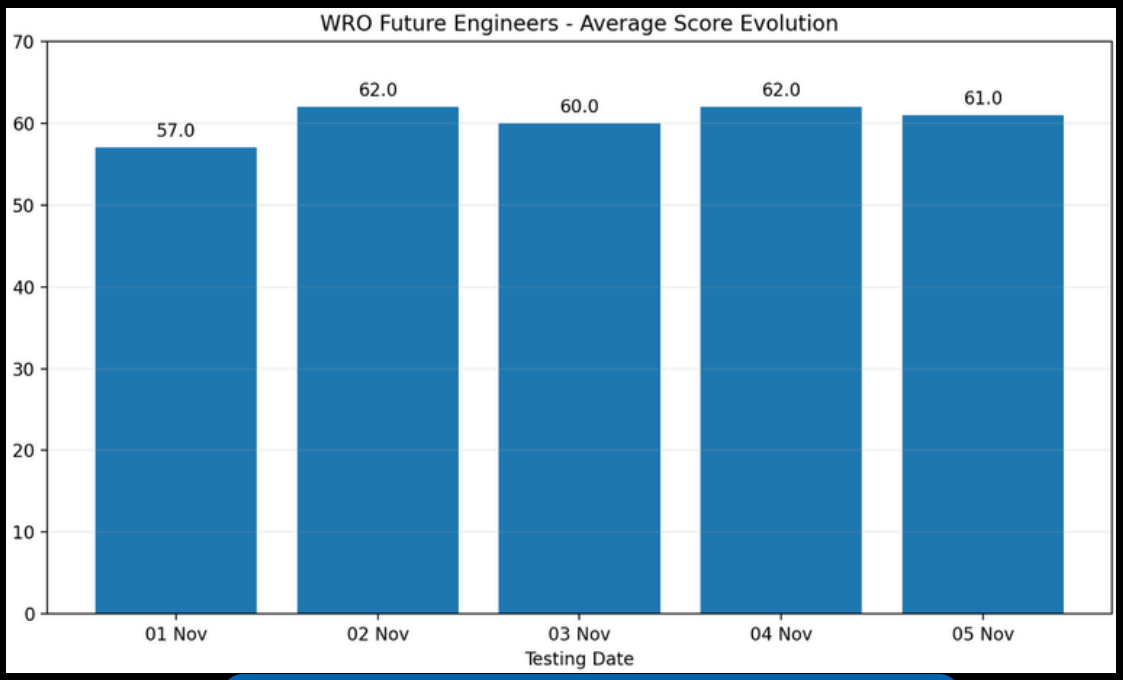


Figure 4.6 - Graph of Scores After Nationals

PERFORMANCE MATRIX

	Color	Points	Time	Issue	Resolved?
6/8/2026	Orange	29	0.23	detected red in ftc field (external)	Yes
6/8/2026	Orange	60	1.11	touched green block	Yes
6/8/2026	Orange	47	1.15	hit pink at start and end both	Yes
6/8/2026	Orange	47	1.15	hit pink in parking	Yes
6/8/2026	Blue	47	1.38	coding issue - wrong function callled	Yes
6/8/2026	Blue	47	1.28	didn't detect last wall roi	Yes
6/8/2026	Blue	47	1.21	moved forward after detecting last wall	No
6/8/2026	Blue	47	1.25	moved forward after detecting last wall	No
6/8/2026	Blue	47	1.09	moved forward after detecting last wall	No

What does this help with?

- **Performance Tracking:** Measures scores and run times to assess consistency.
- **Scenario Comparison:** Compares Orange and Blue runs to identify weaknesses.
- **Error Identification:** Highlights issues with colour detection, movement, and coding.
- **Resolution Tracking:** Tracks fixes
- **Key Weakness:** Repeated wall detection failures show where improvements are needed.
- **Engineering Value:** Guides software changes, improves reliability, and supports future testing.

SECTION 5

Reproducility and Git Quality

Points Covered:

- GitHub structure and clarity
- Commit history (at least 3 commits)
- README structure
- File organization
- CAD / wiring / code files included
- Can another team reproduce this robot?

Points Covered: Github structure and clarity (Section 5), README structure (Section 5), Can another team reproduce this robot? (Section 5), Commit history (Section 5)

GIT OVERVIEW

- All our **code**, **CAD files**, **wiring diagrams**, and this document can be found in our GitHub repository. It can be accessed by scanning the QR code in Figure 5.1 or following this link: *https://github.com/Zkwadia/WRO_FE_Team_Glitch*

Figure 5.1 - QR code to git repository

README Structure

- Our README ("README.md") encapsulates our engineering journey.
- It comprises of **~6400 words** and **~42000 characters**.
- **All 5 Sections:** Our README covers all 5 criteria addressed in this journal, emphasizing on how to reproduce our robot entirely.
- **Reproducibility:** It is completely open-source, allowing any team to take inspiration or reproduce completely.
- **Steps to Reproduce:** It includes detailed steps on how to build, code, wire, and run the robot.

Commit Structure

- **Before Nationals:** Before the national competition, we committed to the git repository multiple times, which 3 main commits (where we achieved set goals on planned dates).
  - These commits happened on 18th June, 20th July and 12th August (2026).
- **After Nationals:** Between nationals and internationals, we committed 3 times (large commits).
  - These commits were done on 7th October, 11th November and 22nd November.

Points Covered: GitHub structure and clarity (Section 5), File organization (Section 5), CAD/Wiring/Code files in Git (Section 5)

GIT STRUCTURE

Folder Structure

- The Git has 5 main folders, called - src, models, schemes, v-photos, and other. These folders hold content such as CAD files, codes and wiring diagrams.
  - models/ - holds all **CAD** (STEP and STL) files to be 3D printed, laser-cut, or CNC cut. Also holds DXF dimension drawings
  - src/ - holds all **code** files
    - /OpenCV - holds calibration files
    - /ESP - holds the ESP32 reading and communication code
    - /Pi - holds all Raspberry Pi code - main files and test scripts
  - schemes/ - holds calculations (speed, torque, current, etc.) and **wiring** diagrams
  - v-photos/ - robot photos, including all 6 angles and isometric
  - other/ - holds images of parts, specifications, and diagrams

- There are also multiple other folders:
  - document/ - holds this engineering document
  - logs/ - holds sample outputs from open and obstacle runs
  - service_files/ - holds the service scripts to run codes on boot
  - t-photos/ - holds photos of our team - official and fun
  - versionTest/ - holds test scripts for subsystems

Points Covered: - Code documentation (Section 3), GitHub structure (Section 5), GitHub clarity (Section 5)

STEP 1 - 3D PRINT THE CHASSIS

All the CAD and structural files are attached in the models folder of our Github Repository, in the structure shown below.

```
odels/
— 3D prints/                              # STL and project files — slice and print directly
   ├— all part 3d print file.3mf          # Complete multi-part print project (open in Bambu/PrusaSlicer)
   ├— project file.3mf                    # Alternate slicer project
   ├— new camera mount.STL               # Camera mount with 10° downward tilt
   ├— long stick.STL                     # Vertical sensor mast
   ├— long stick mount.STL               # Base mount for sensor mast
   ├— front cover.STL                    # Front bumper cover
   ├— battery 1.STL                      # Battery holder
   ├— an_6_12_holder.STL                 # 6mm/12mm rod holder
   ├— an_spacer internationale.STL       # Spacer for international chassis spec
   ├— spacer with tf mount.STL           # Spacer with sensor bracket
   ├— tf mini mount horizontal.STL       # Horizontal sensor bracket (legacy, for reference)
   ├— atfmini mounting.STL               # Alternate sensor mounting bracket
   ├— rp lidar mount.STL                 # RPLidar mount (legacy, for reference)
   └— an waveshare 5 inch dsi lcd.STL    # Display mount (optional)
— STEP/                                  # Editable CAD files (SolidWorks, Fusion 360, OnShape, FreeCAD)
   ├— WRO FE Car complete CAD.STEP        # Full robot assembly — start here
   ├— chassis_Polycarbonate_6mm thickness_qty1pc.STEP   # Chassis plate (cut from 6mm polycarbonate)
   └— electronics Mount_3mm polycarbonate_ qty 1.STEP   # Electronics tray (cut from 3mm polycarbonate)
```

Important Notes:
The chassis and electronics tray are laser-cut or CNC-cut from polycarbonate sheets. All files in the 3D prints folder are printed parts. The .3mf files contain pre-arranged plates with all parts oriented.

Recommended Print Settings:

Parameter	Written By
Material	PETG or PLA+ (PETG prefferred)
Layer Height	0.2 mm
Infill	40% gyroid
Perimeters / Walls	4
Supports	Required for “new camera mount.STL” and “long stick mount.STL” only
Bed Adhesion	Brim for any part with a narrow base

Tip for Assembly:
Open WRO FE Car complete CAD.STEP in your CAD tool to see how every part fits together before printing or cutting anything.

STEP 2 - LASER CUTTING...

- Refer to 'WRO FE Car complete CAD.STEP' as it shows all of the components in their final, installed position.
- For laser-cutting the polycarbonate chassis, and deadweight, .STEP files are provided. Refer to the following drawings.

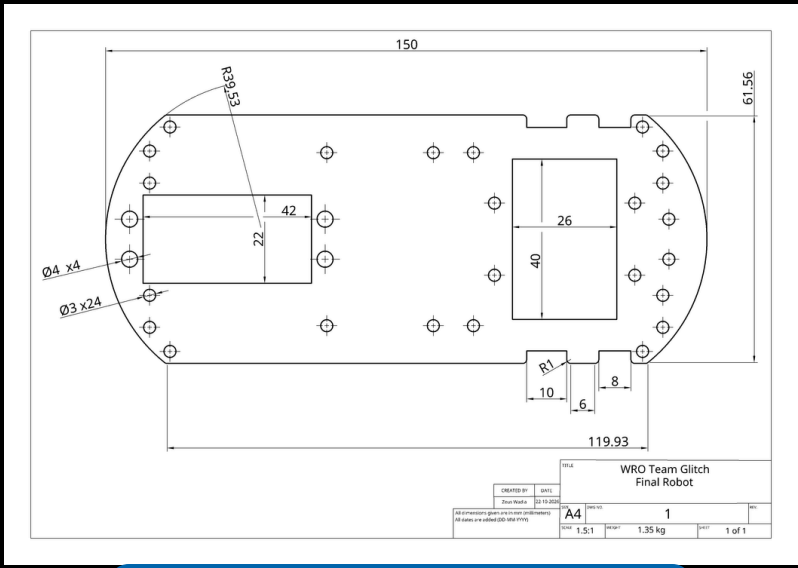


Figure 5.2 - Polycarbonate Chassis

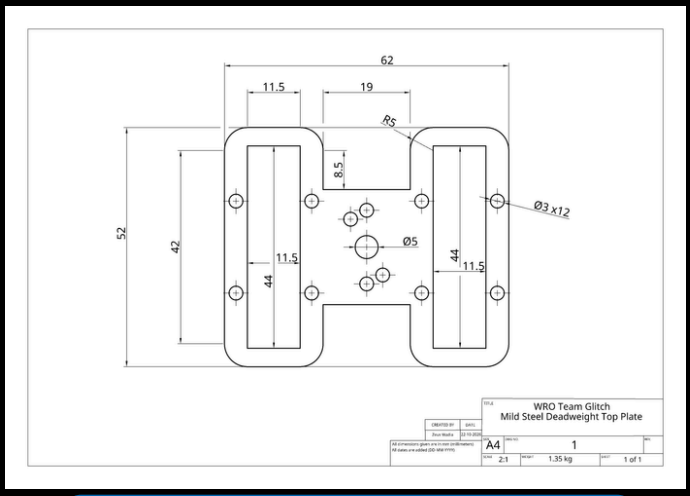


Figure 5.3 - Deadweight Top Piece

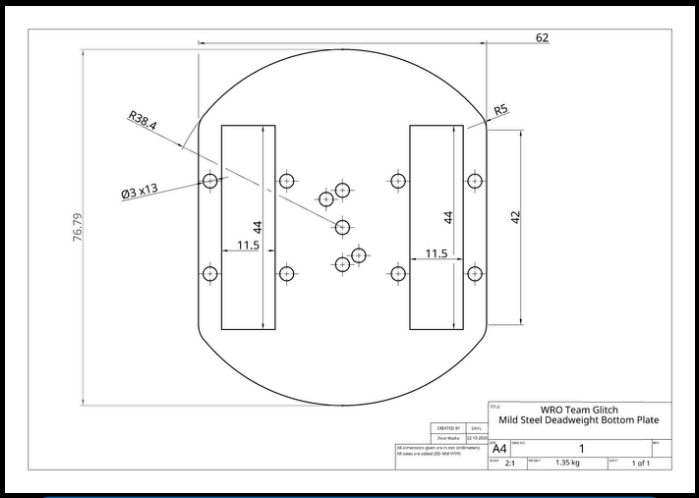


Figure 5.4 - Deadweight Bottom Piece

- Cut the polycarbonate chassis (6mm polycarbonate chassis.dxf) out of 6mm polycarbonate. Cut the electronics tray (electronics Mount_3mm polycarbonate_ qty 1.STEP) out of 3mm polycarbonate. Cut the deadweights (3mm mild steel deadweight upper.dxf **AND** 3mm mild steel deadweight lower.dxf) out of 3mm mild steel.

Points Covered: CAD files included (Section 5), Can another team reproduce this robot? (Section 5)

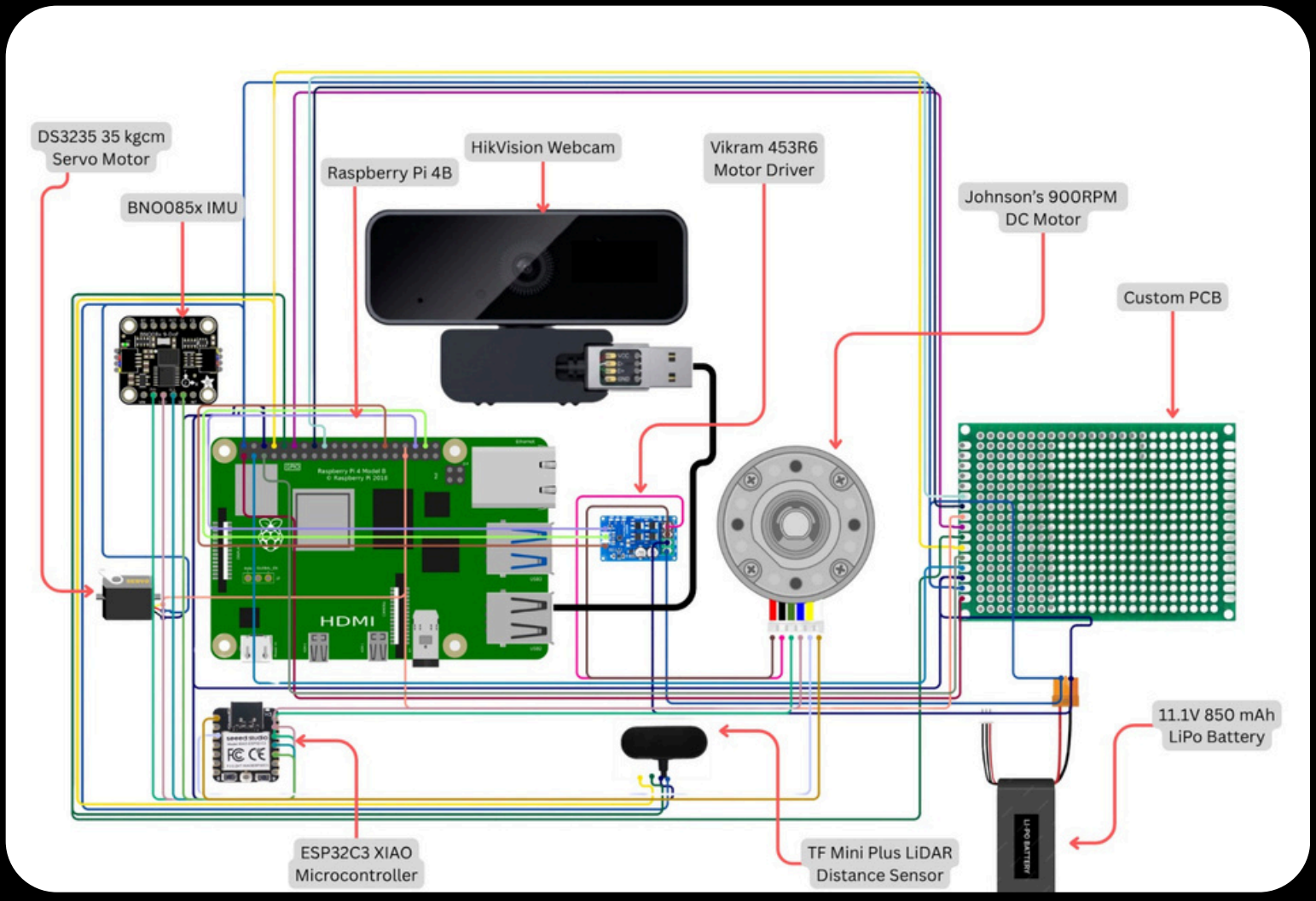
STEP 3 - MECHANICAL ASSEMBLY

- **Drive motor:** Mount the Johnsons 900RPM DC motor into the rear motor bay using M3 bolts. Route the encoder cable forward along the chassis underside.
- **Steering servo:** Press the DS 35kg·cm servo into the front servo bay and connect to the Ackermann steering linkage.
- **Front axle:** Install the Ackermann steering linkage. Both front wheels should turn at slightly different angles at full lock - verify the inner wheel turns sharper than the outer wheel.
- **Sensor mast:** Print long stick.STL and long stick mount.STL. Mount the mast vertically at the front-centre of the chassis.
- **Camera mount:** Print new camera mount.STL and attach to the top of the sensor mast or front rail. The mount has a fixed 10° downward tilt - no adjustment needed.
- **Front cover:** Print front cover.STL and clip onto the front bumper rail to protect sensor wiring.
- **Battery holder:** Print battery 1.STL and secure the LiPo pack to the chassis deck.
- **Electronics tray:** Screw the cut 3mm polycarbonate tray to the chassis deck. Mount the Raspberry Pi, XIAO ESP32-C3, and MDD3A onto the tray using the pre-drilled mounting bosses visible in the STEP file.

STEP 4 - ELECTRONICS WIRING

Wire all components according to the wiring diagram in schemes/wiring_diagram.png.

Below is the diagram for the power rails.



Critical Wiring Rules:

- The BNO085 must be powered from the XIAO's 3V3 rail, not the Pi's rail. Sharing the Pi rail introduces motor EMI into the IMU heading.
- Run the motor wires and encoder wires on opposite sides of the chassis - motor current induces EMI in nearby encoder signal lines.

Points Covered: Code files included (Section 5), Can another team reproduce this robot? (Section 5)

STEP 5 - ESP32 FIRMWARE

1. **Install *Arduino IDE (≥2.0)* and add the Seeed Studio board package:**
   - In Arduino IDE: File → Preferences → Additional boards manager URLs
   - Add: https://files.seeedstudio.com/arduino/package_seeeduino_boards_index.json
   - Tools → Board → Boards Manager → search "Seeed XIAO ESP32C3" → Install

2. **Install required libraries via Library Manager:**
   - Adafruit BNO08x
   - Adafruit Unified Sensor

3. **Open src/xiao/imu_encoder.ino, select board XIAO ESP32C3, select the correct COM/tty port, and upload.**

The firmware reads BNO085 Euler heading over I2C and quadrature encoder counts via GPIO, then broadcasts over USB UART at 115200 baud in the format:
<heading_float> <encoder_int>\n

Serial commands accepted at runtime:

- b"R" - software reset (reboots XIAO and reinitialises BNO085)
- b"1" - zero the encoder count

Points Covered: Code files included (Section 5), Can another team reproduce this robot? (Section 5)

STEP 6 - RASPBERRY PI SETUP

OS: Raspberry Pi OS 64-bit Bookworm (Lite or Desktop). Enable I2C and the camera interface via raspi-config.

- Install system dependencies:
  - *sudo apt update && sudo apt install -y python3-pip pigpio python3-pigpio ffmpeg i2c-tools*

Install Python packages:
pip3 install opencv-python pyserial RPi.GPIO adafruit-circuitpython-vl53l0x numpy

Verify I2C bus and sensors:

- # Start pigpio daemon
  - *sudo pigpiod*

Set up USB device alias so /dev/XIAO_USB always resolves to the XIAO (plug the XIAO in first):

- # Find vendor and product ID
  - *lsusb # note idVendor and idProduct for the XIAO# Create udev rule*
- *sudo nano /etc/udev/rules.d/99-wro.rules*
- *Add this line (replace XXXX/YYYY with your VID/PID from lsusb):*
- *SUBSYSTEM=="tty", ATTRS{idVendor}=="XXXX", ATTRS{idProduct}=="YYYY", SYMLINK+="XIAO_USB"*
- *sudo udevadm control --reload-rules && sudo udevadm trigger*

Clone the repository:

- *cd /home/pi*
- *git clone https://github.com/YOUR_TEAM/WRO_2026.git*
- *cd WRO_2026/versionTest*

STEP 7 - CONFIGURE PID GAINS

Edit gains.json before the first run. Start conservative and tune iteratively - the file hot-reloads while the script is running so you do not need to restart between adjustments:

```
{
 "kp": 0.6,
 "kd": 0.01,
 "ki": 0,
 "kp_s": 0.5,
 "kd_s": 0.5,
 "kp_v": 0.01,
 "ki_v": 0,
 "kd_v": 0,
 "SPEED_SCALE": 2447.0
}
```

kp/kd/ki - heading PID (servo). kp_s/kd_s - centroid/wall PD (servo). kp_v/ki_v/kd_v - speed PID (motor duty, obstacle challenge only). SPEED_SCALE - encoder counts per second at 100% duty on a fresh battery; calibrate this from your first run logs before enabling ki_v or kd_v.

Points Covered: Code files included (Section 5), Can another team reproduce this robot? (Section 5)

STEP 8 - RUN

Open the terminal on the Raspberry Pi and run:

```
cd /home/pi/WRO_2026/versionTest
sudo python3 open_challenge.py
```

```
cd /home/pi/WRO_2026/versionTest
sudo python3 obstacle_3.py
```

This is our path, either copy our repo or use your own and edit the path

Both scripts start pigpiod automatically. Logs are written to /home/pi/wro_logs/logs/ with timestamps. Flip the kill switch (GPIO 12) to start/stop the robot without restarting the script.